# Kubernetes is too complex for startups

# Kubernetes is too complex for startups

# WRONG!

# Why this talk?

# How many times have you heard ?

*“Kubernetes is only for enterprise”*

*“Kubernetes is too complex”*

*“We can always migrate to kubernetes later”*

*“Kubernetes is not right for us”*

*“We don’t need that kind of flexibility”*

*“We are special”*

*“Our product is simple”*

# If your product is simple …

# If your product is simple …

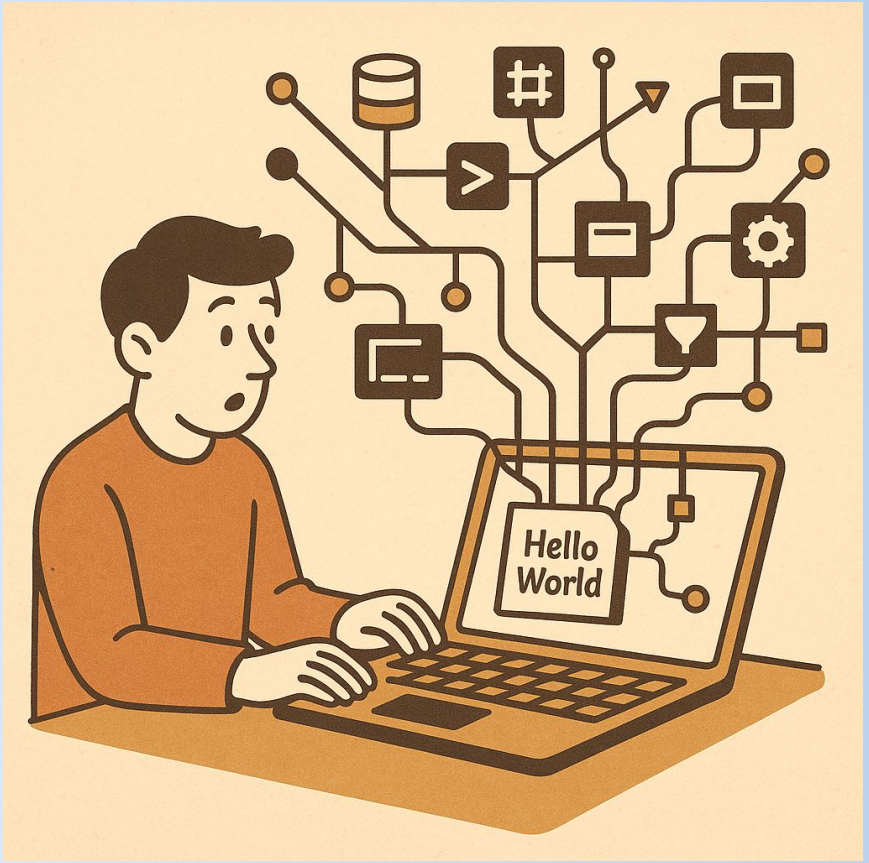

## … it’s still just a prototype.

# Building a business *is* complex.

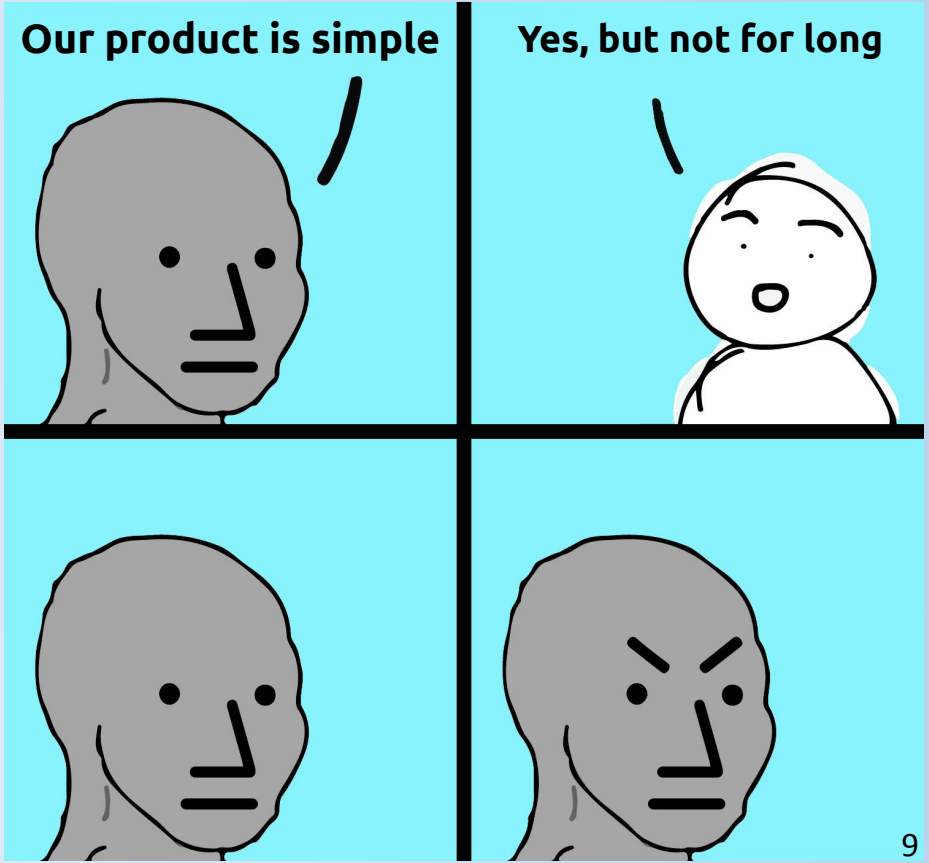
# Building a business *is* complex.

&

# Building software *is* complex.

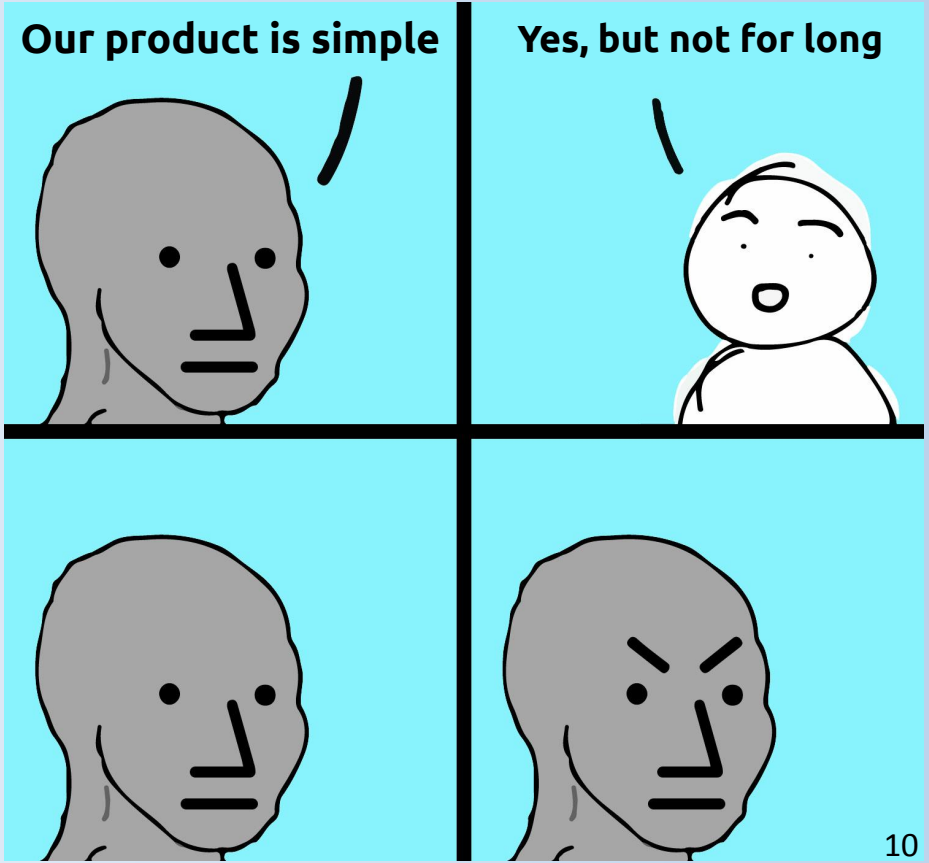


# You *will* have to deal with complexity!



# You *will* have to deal with complexity!

*So the question is …*



# How do you choose the right technology to handle growing complexity?

**You choose modern, but *boring* technologies.**

Dan McKinley: https://mcfunley.com/choose-boring-technology

Boring technologies are **well known**, **mature** and have **wide community+industry support**.

Choosing mature technologies makes it:

- Easier to do the thing you want
- Easier to debug
- Easier to hire for
- Easier to find documentation
- Easier to …

Don’t risk experimental technologies, make the safe choice!

**Whenever you do something complex, you spend your ‘innovation tokens’.**

**Make sure to spend them on what matters: The business!**

**Not on fighting complexity.**


https://boringtechnology.club/#17

# Rhetorical Question:

# “What do dating (apps) and energy trading have in common?”

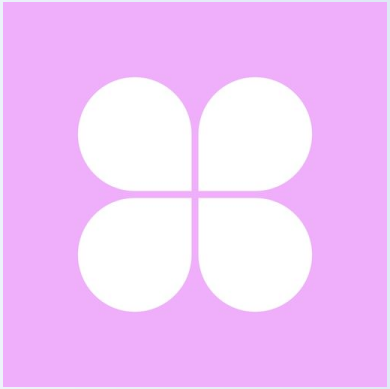
# Rhetorical Question:

**“What do dating (apps) and energy trading have in common?”**

**Both requires software.**

# … And making software is complex.

# This talk is based on our experience dealing with complexity using Kubernetes, in startups, in two very different fields



Dating App


Energy Trading

# Zander

- 2y @ Green.ai
- Head of Software Platform
- Consulting and Teaching about K8s
- Organizer @ Cloud Native CPH
- I run K8s “for fun”
  in my homelab

# Dag

- Platform team in Egmont
- Spend most of my career focusing on Kubernetes and GitOps

EGMONT

2 EGMONT

## TV Streaming

2 Play

## Platform

2 VIMOND RiksTV

story house EGMONT

## Agencies

OKTO · S360 · CORE · The Story Laboratory · ingager · MarkedsPartner · KLINTBERG NILÉN

## Magazines

Journal · Hus&Hem · EUROWOMAN · EUROMAN · HER OG NÅ · talk town · story house EGMONT · ALT · PEOPLE EGMONT · FLIPP · ICAKURIREN · ANDERS AND

## E-commerce

jollyroom.se · Bagaren & Kocken · med24.dk · RoyalDesign · outnorth · GARNIUS · KitchenOne · Fjellsport · SKITT FISKE

NORDISK FILM EGMONT

## Distribution

NORDISK FILM · PlayStation

## Production

SHORTCUT · DANSK REKLAME FILM · NORDISK FILM · ZENTROPA · MEDIA DIRECT NORGE · DRIVE

## Cinema

NORDISK FILM BIOGRAFER · NORDISK FILM KINO

## Gaming

Avalanche Studios Group · NITRO · Star Stable · flashbulb games · SUPERMASSIVE GAMES

EGMONT CAPPELEN DAMM

## Digital books

SAGA

CAPPELEN DAMM

## Education

Alinea · Akademisk Forlag · GoTutor · Praxis

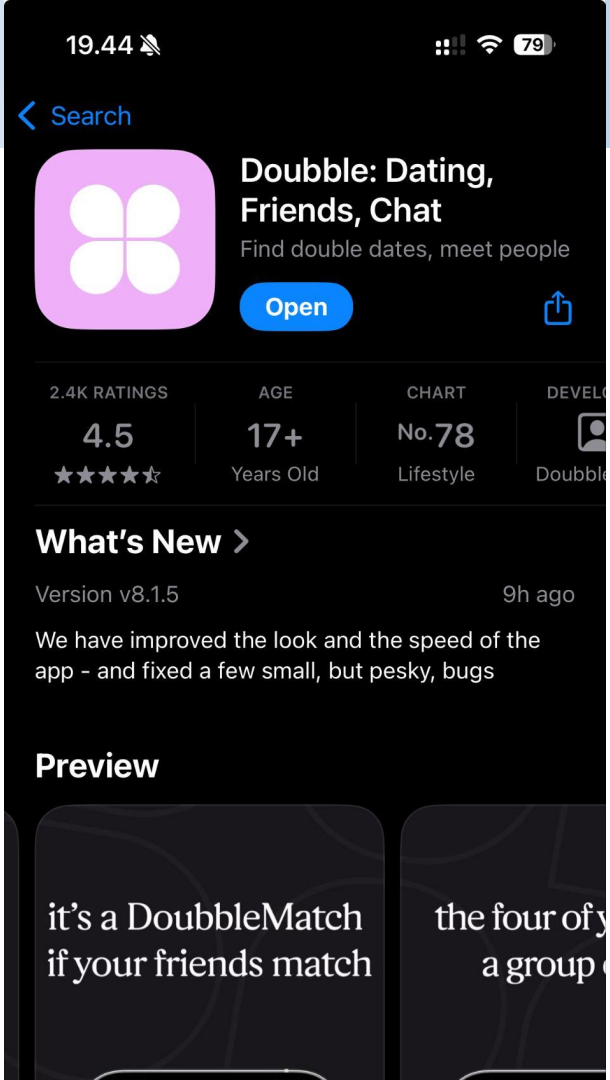
We bring stories to life

# Dag

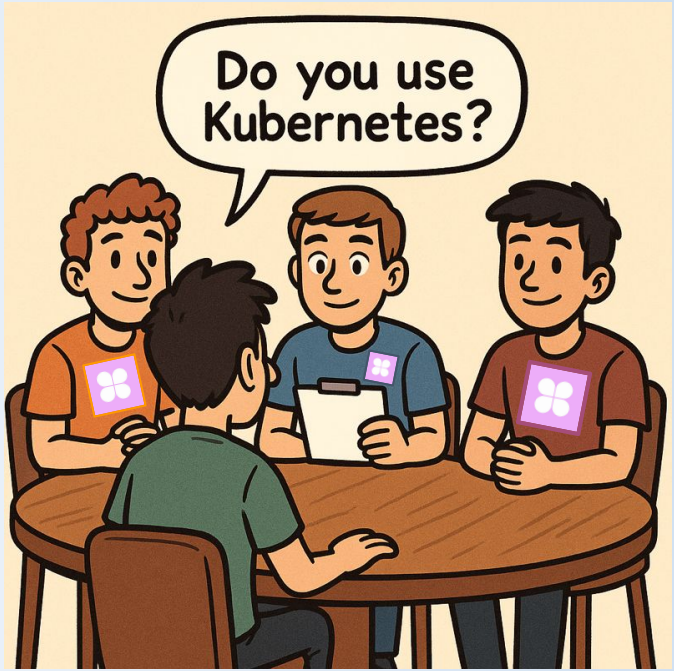
- From 2024 to 2025
- 5 people
- Main person responsible for all infrastructure

# Doubble

- Dating App
- 5 people in the company
- Mainly popular among 18–22 year olds



# When i joined Doubble



**“We are only running a few containerized services, so we don’t need Kubernetes”**

**“We don’t expect to run more services in the near future**

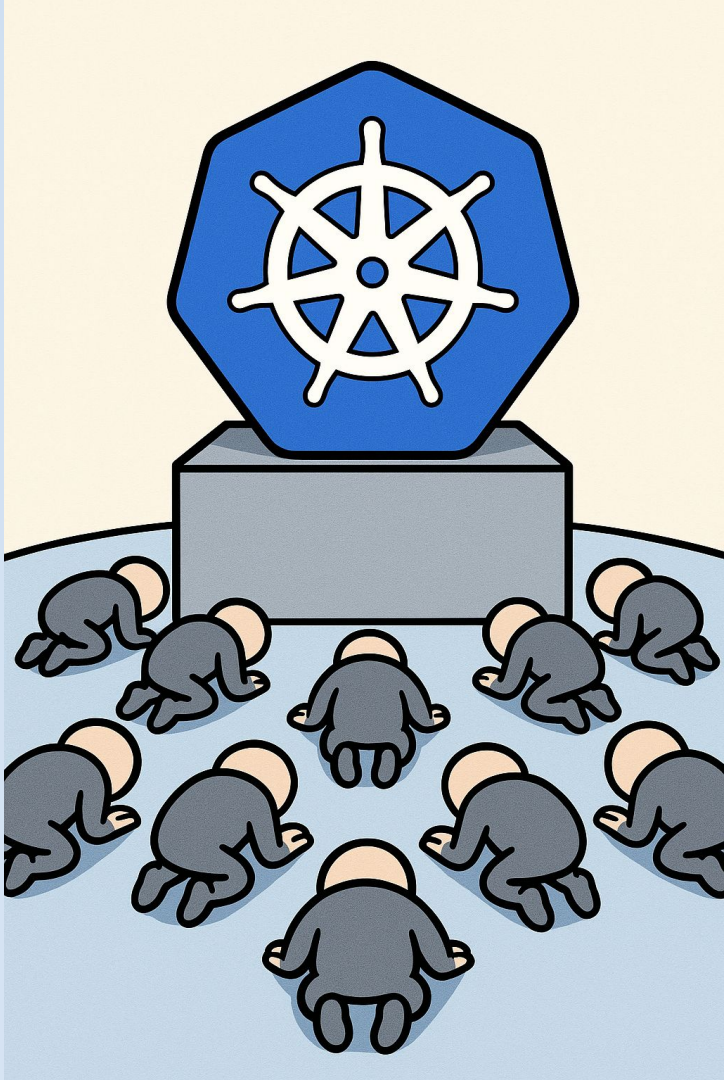
# Maybe they are right!

# Maybe Kubernetes is not the solution to every problem?

**Maybe i only have a hammer, so all i see is nails?**

**Maybe i should expand my tool box?**

# Maybe i have spend too much time at Kubernetes Conferences??



AWS
ECS

AWS
Fargate

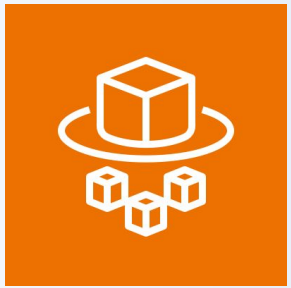
Google
Cloud
Run

Azure
Container
Apps

Azure
Container
Instances

# Serverless compute engine

Lets you run containers without managing the underlying servers



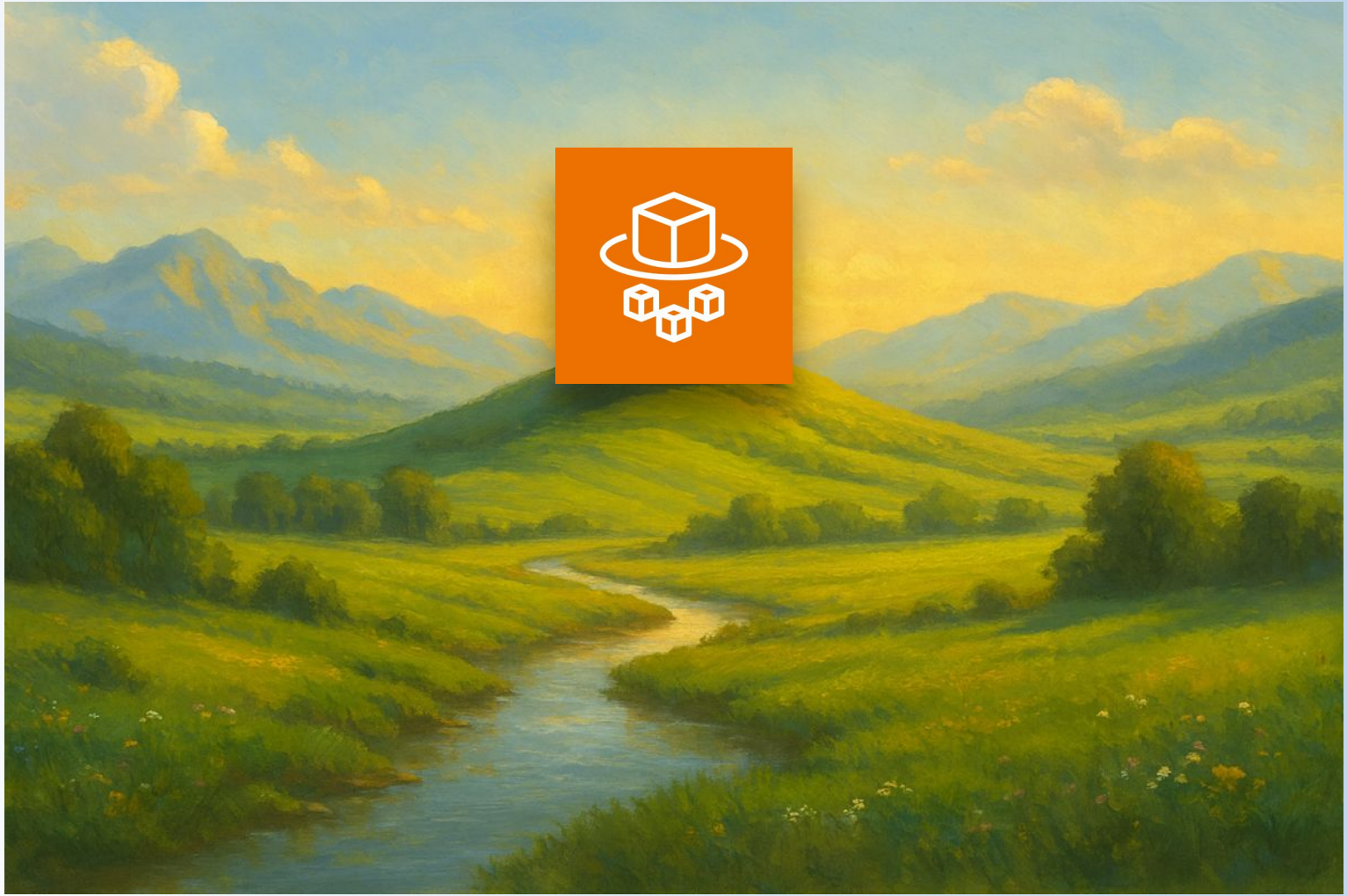
AWS
Fargate

**Easy to get started.**

**Easy to work with.**

**No need to worry about infrastructure or orchestration.**


CONTRACT



# AWS Fargate on ECS

**You still need to define:**

- **Environment variables**
- **Secrets**
- **CPU & Memory**
- **Health check endpoints**
- **Open ports/paths**
- **Some kind of rollout strategy**
- **Autoscaling policies**
- **Service Discovery**

# AWS Fargate on ECS

**You still need to define:**

- **Environment variables**
- **Secrets**
- **CPU & Memory**
- **Health check endpoints**
- **Open ports/paths**
- **Some kind of rollout strategy**
- **Autoscaling policies**
- **Service Discovery**

**Does this remind you of something?**

# Realization #1:

# Declaring workload is *not* simpler when using Fargate/serverless

# Hmm… Let’s take a look at the architecture diagram…

Cloudflare

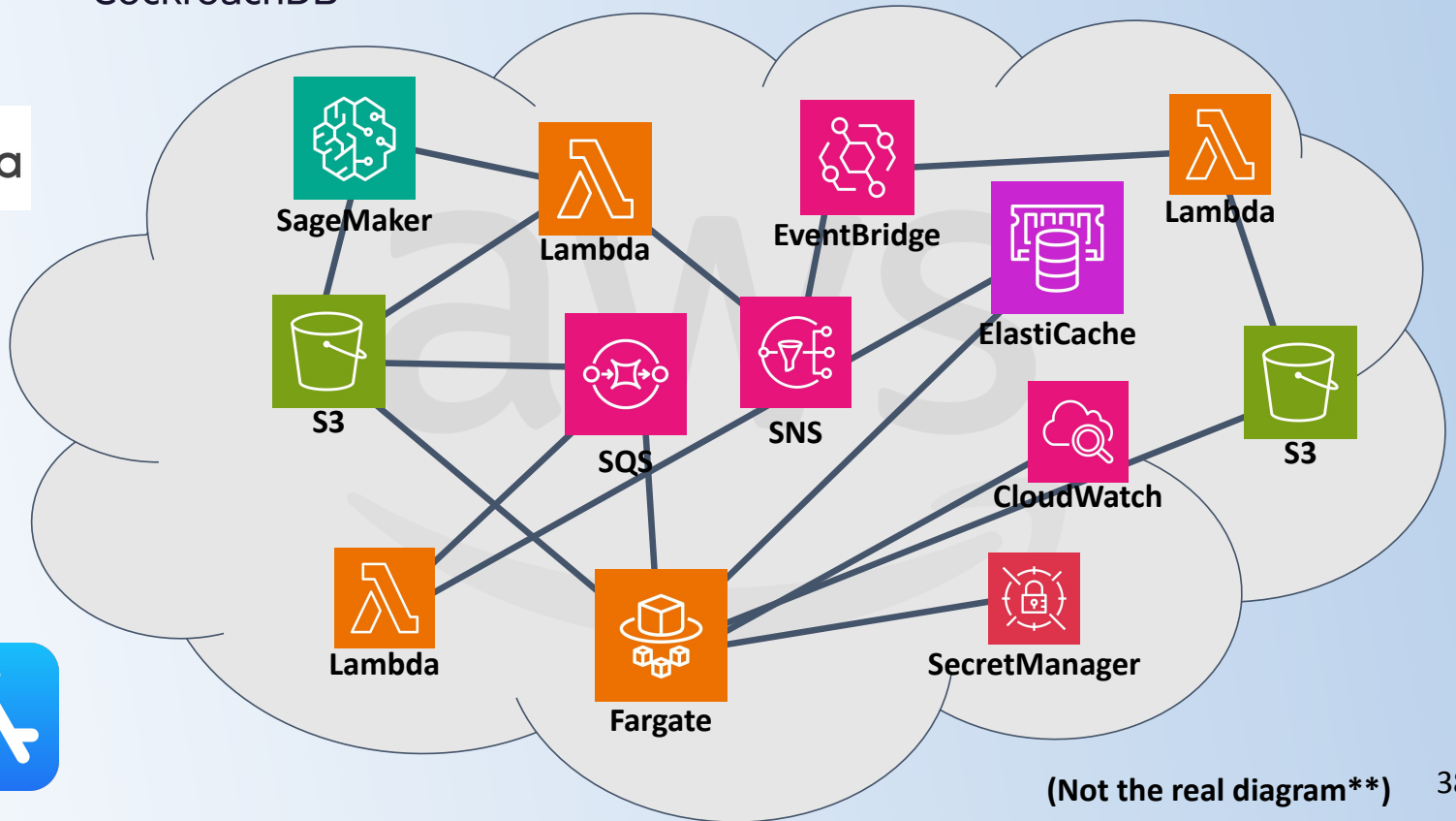
CockroachDB

ClickHouse

Google Play

Grafana

SENTRY




(Not the real diagram**)

Cloudflare

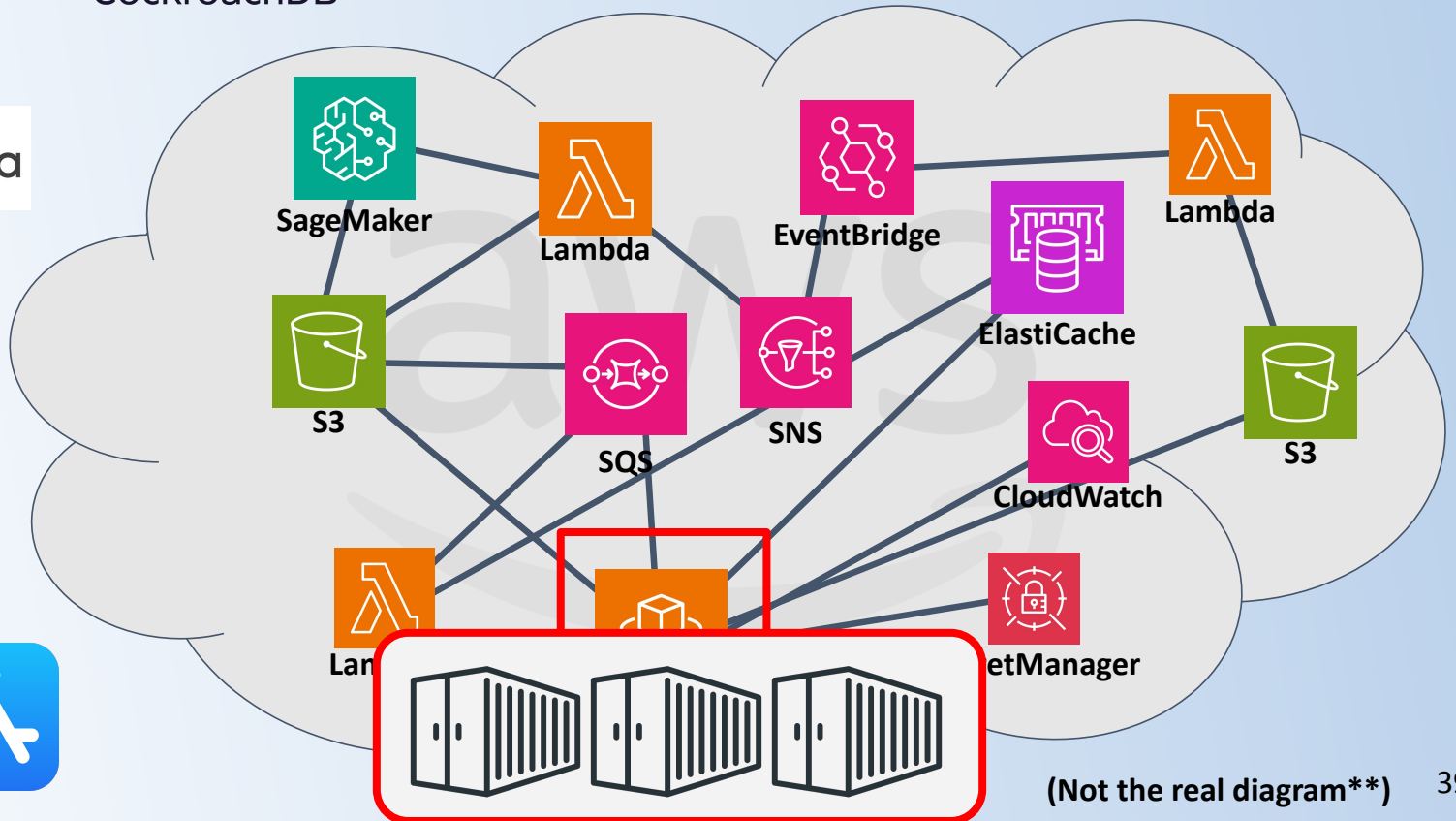
CockroachDB

ClickHouse

Google Play

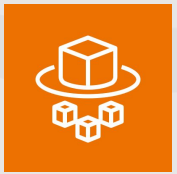
Grafana

SENTRY

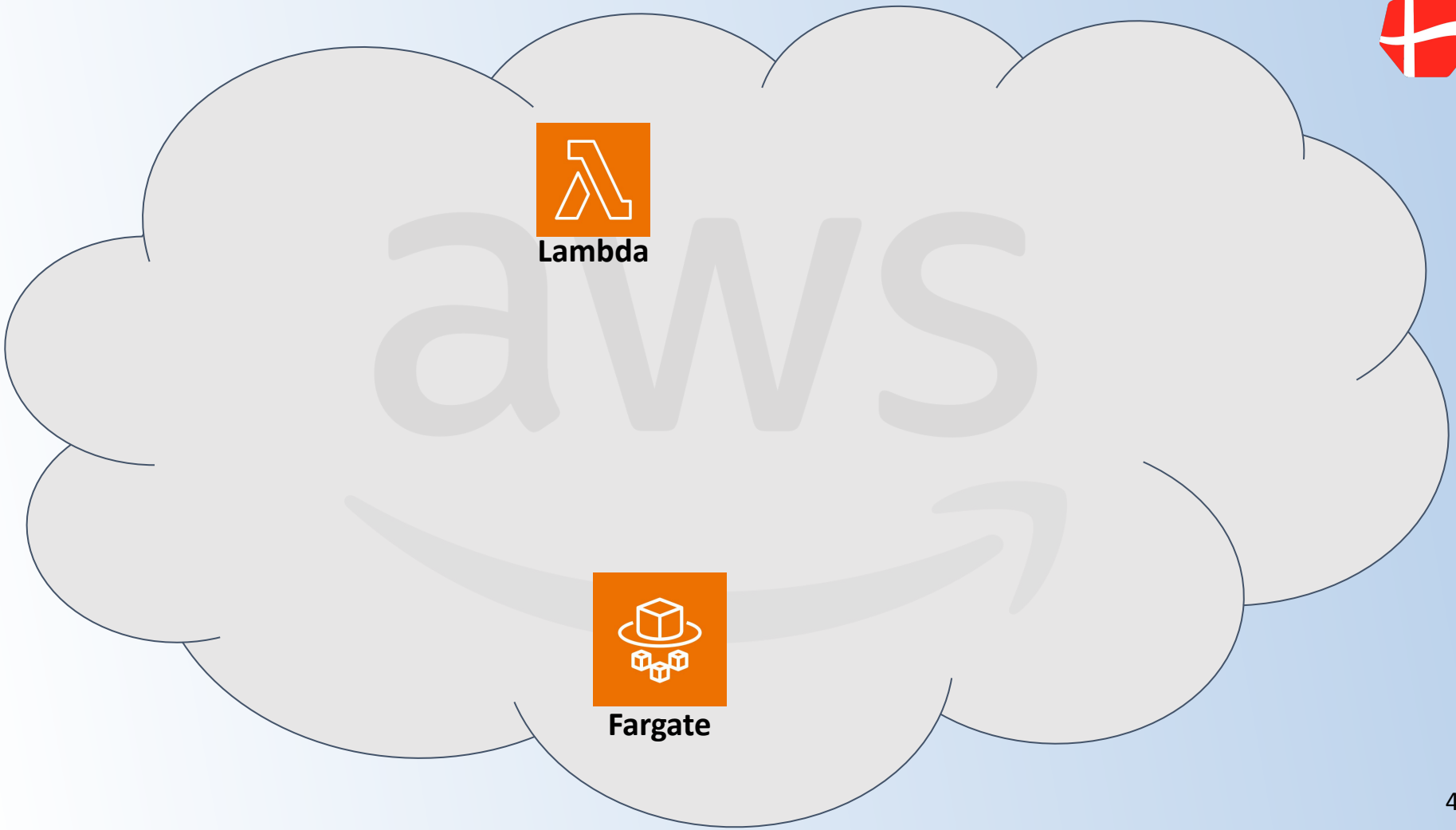


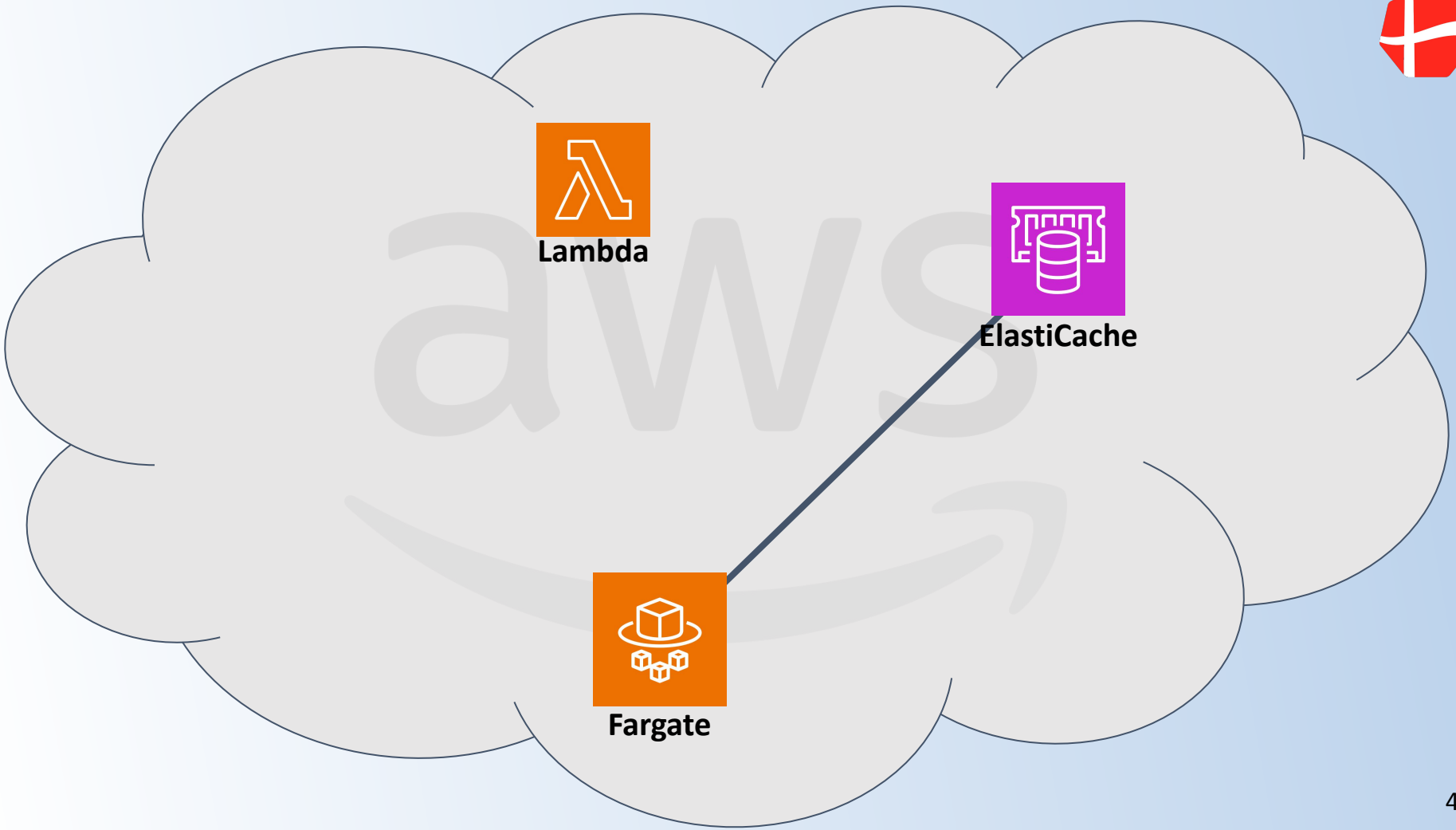

(Not the real diagram**)

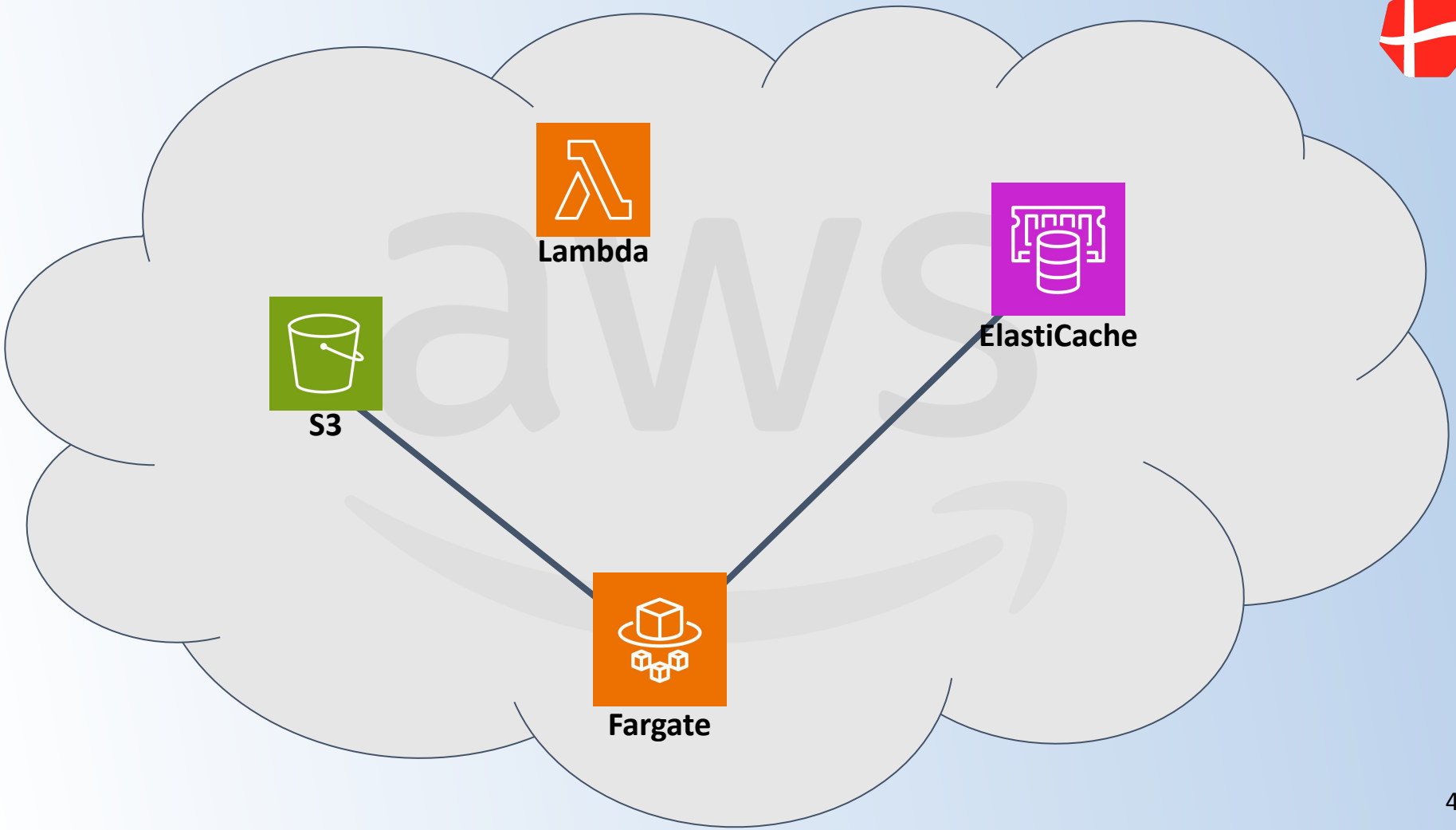
# But how does that happen?

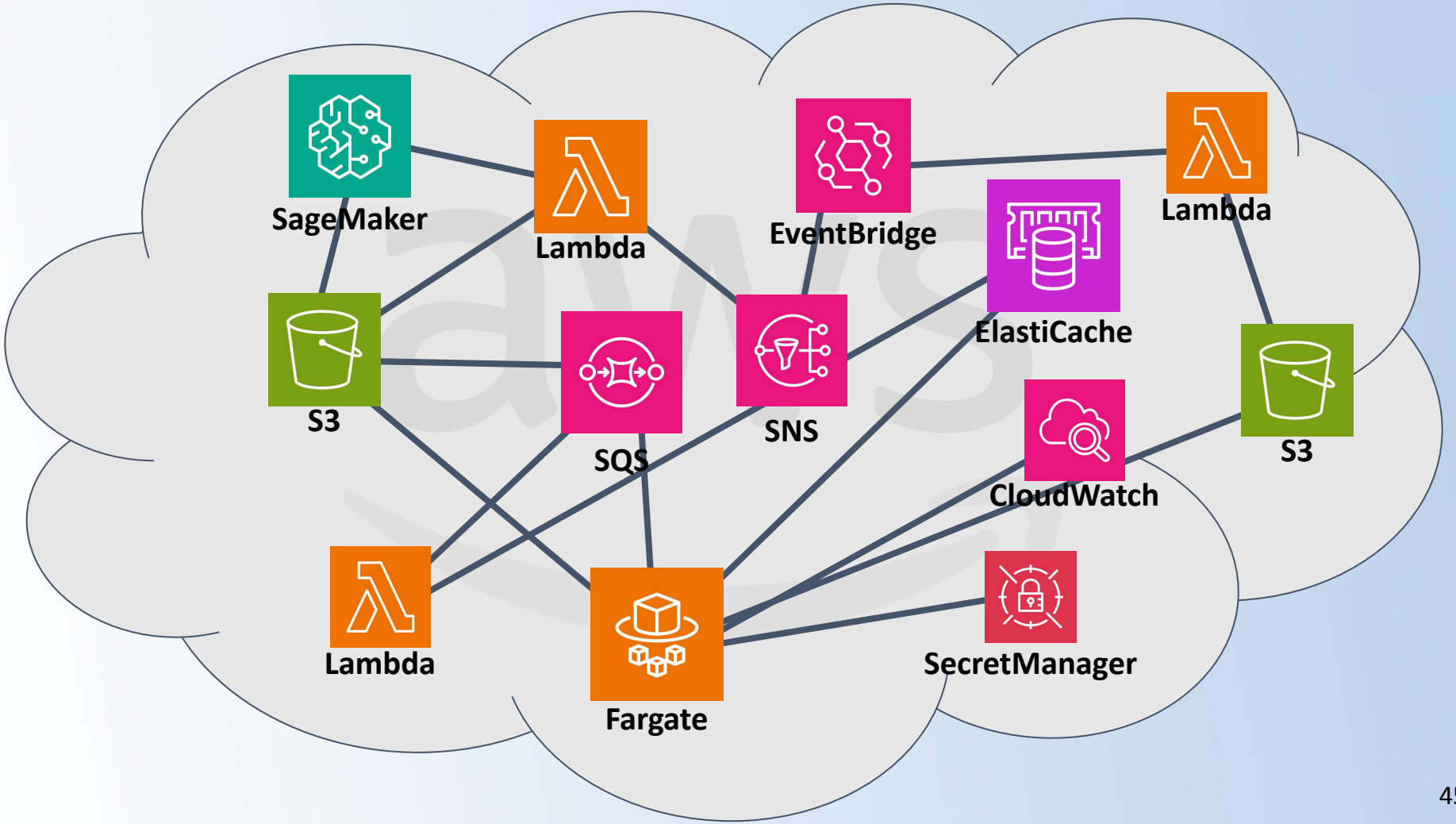


Fargate


Lambda
Fargate


Lambda
ElastiCache
Fargate


Lambda
S3
ElastiCache
Fargate

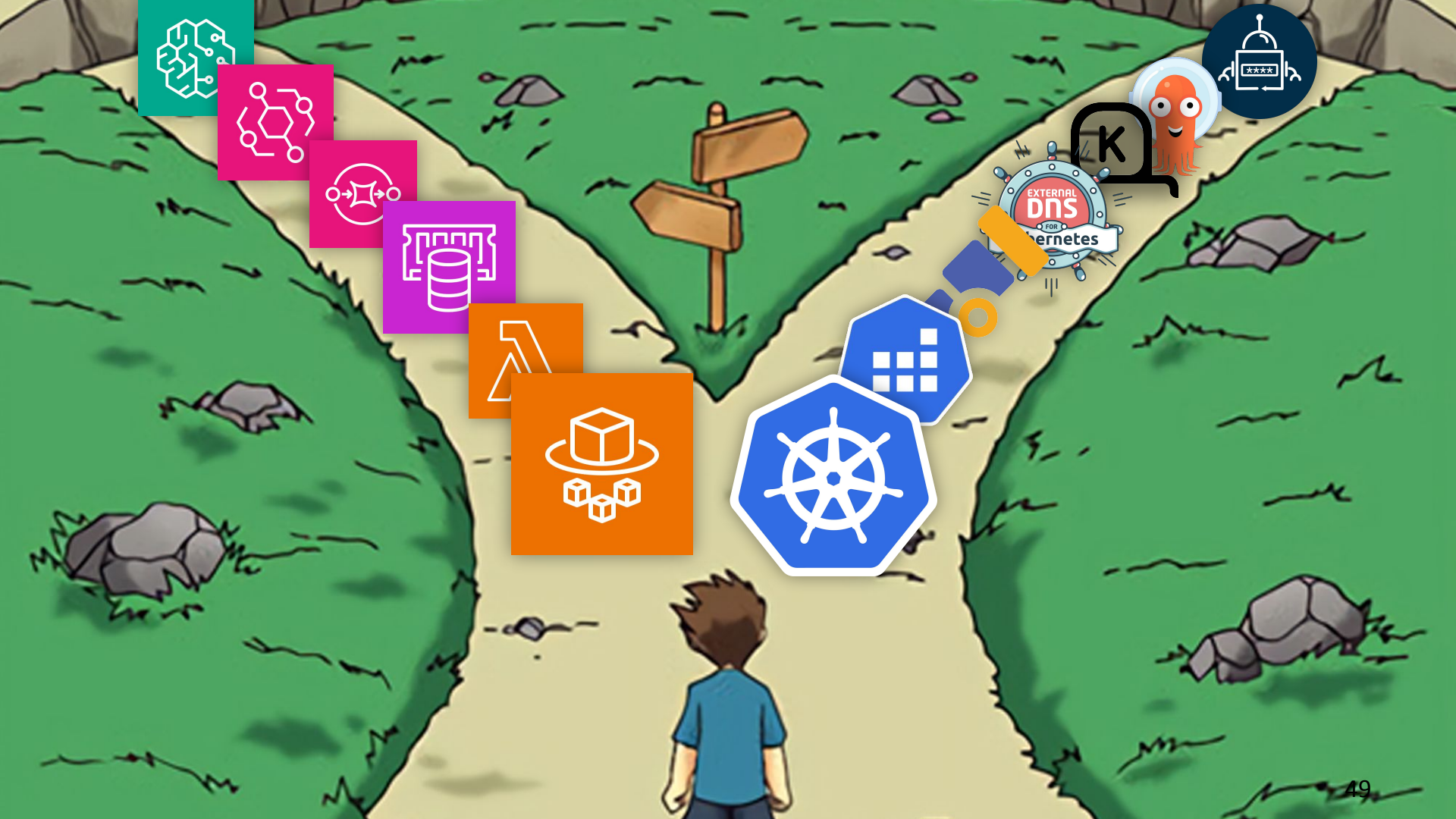

SageMaker
Lambda
EventBridge
Lambda
S3
SQS
SNS
ElastiCache
S3
CloudWatch
Lambda
Fargate
SecretManager


SageMaker
Lambda
EventBridge
Lambda
ElastiCache
S3
SQS
SNS
S3
CloudWatch
Lambda
Fargate
SecretManager


All of these tools are simple on their own.
But together, they become incredibly complex.
S3
Lambda
Fargate
SecretManager

**BUT! This isn’t because of poor design…**

**It’s simply the natural consequence of choosing a serverless solution.**


EXTERNAL
DNS
FOR
ernetes

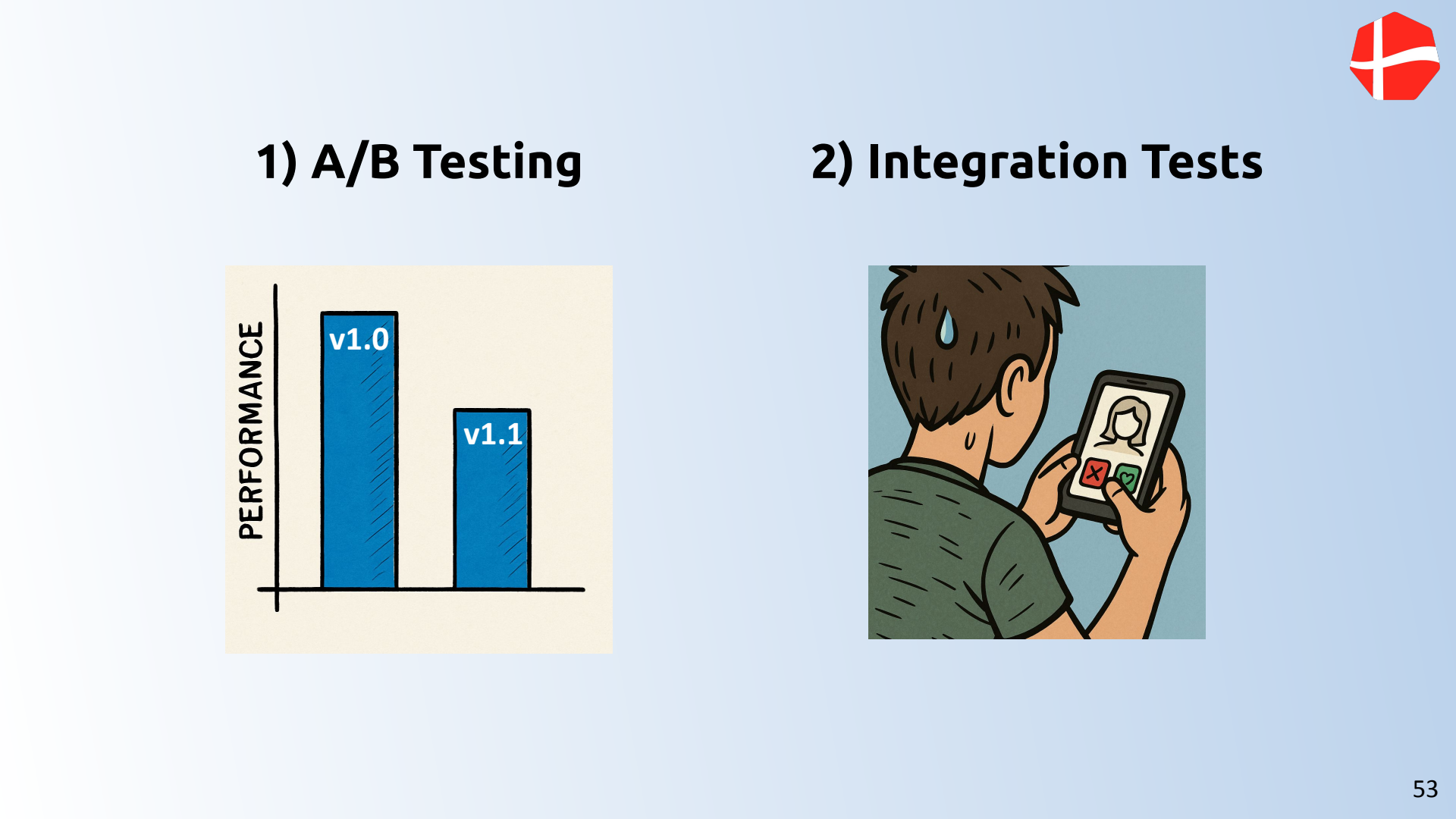
# Realization #2:

# Starting with serverless solutions (e.g., Fargate) has strong implications on your overall architecture.

**But okay… So far:**

- **I am not impressed with the way you declare workload**
- **And i am not impressed with the implications serverless has on your overall architecture**

**That was my initial assessment, but now the real work begins…**



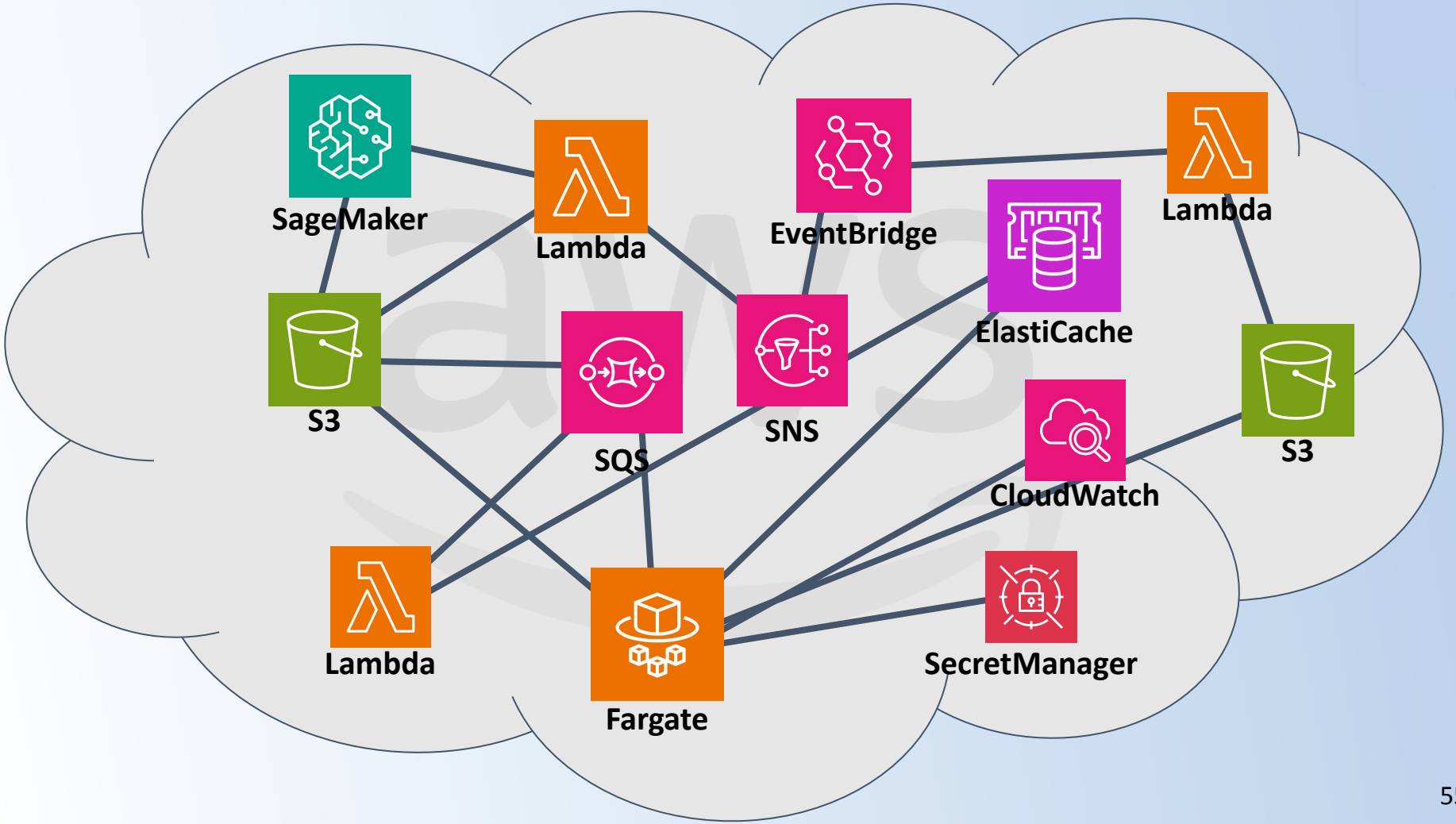
# 1) A/B Testing

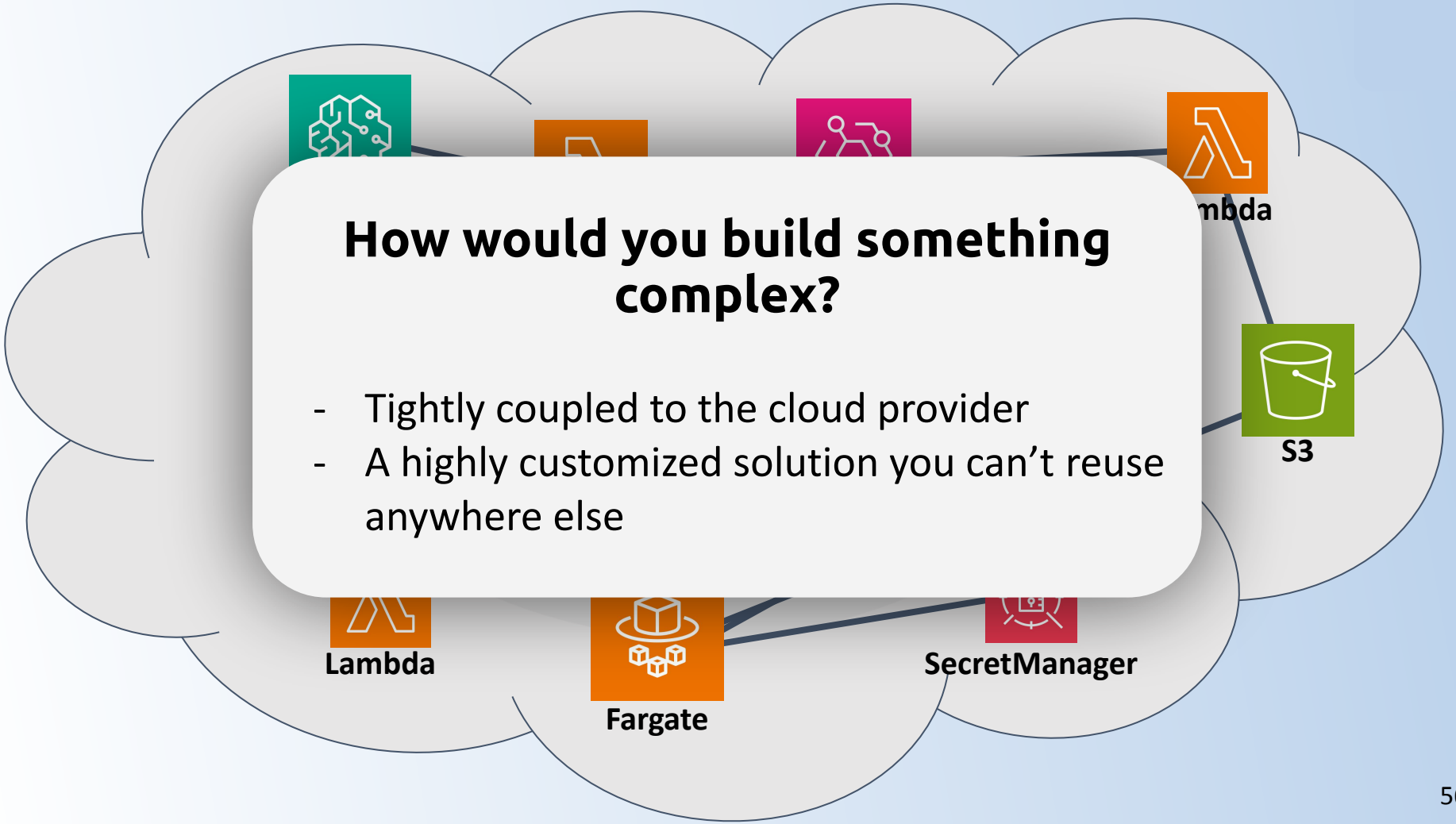


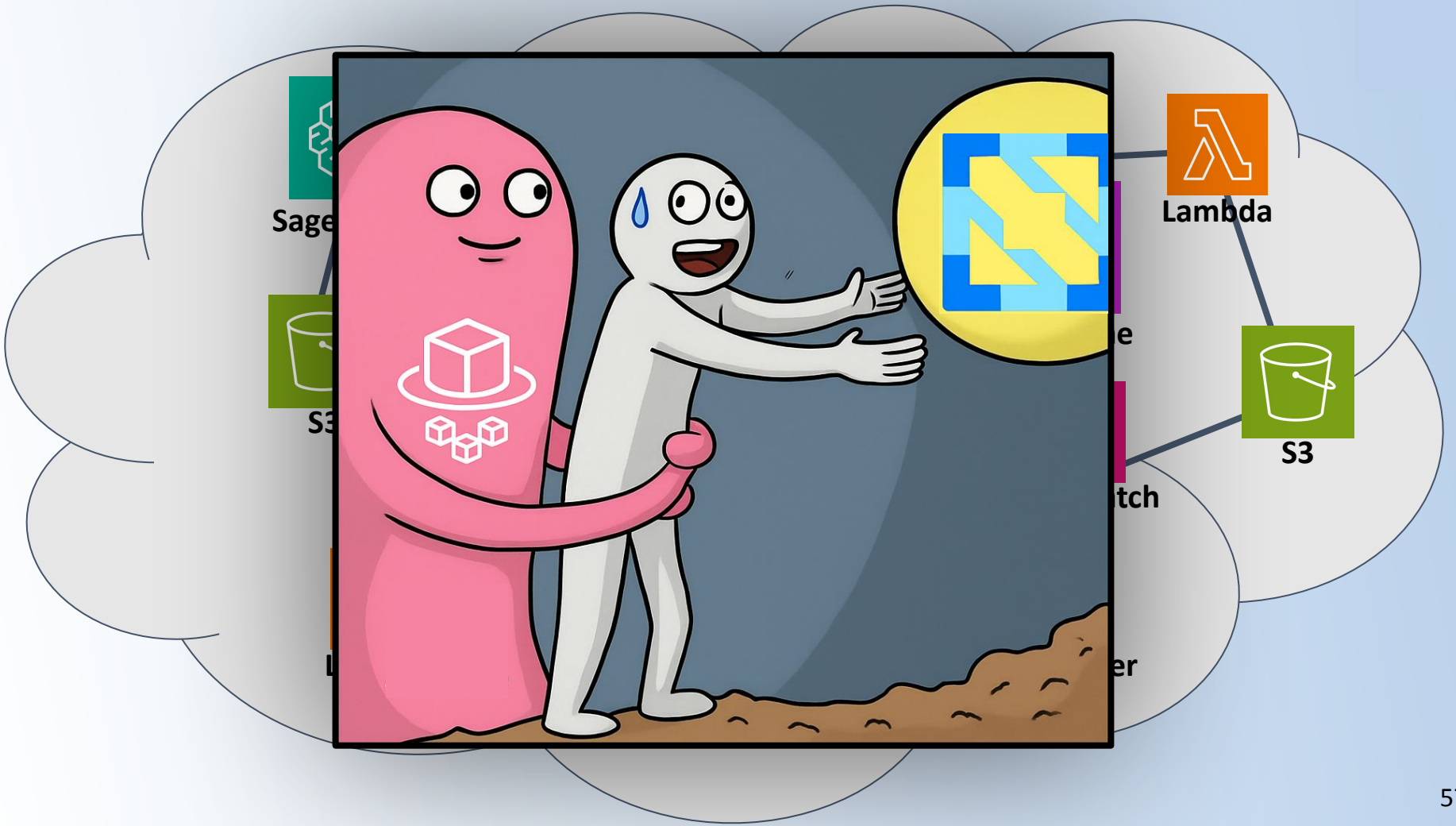

# 2) Integration Tests

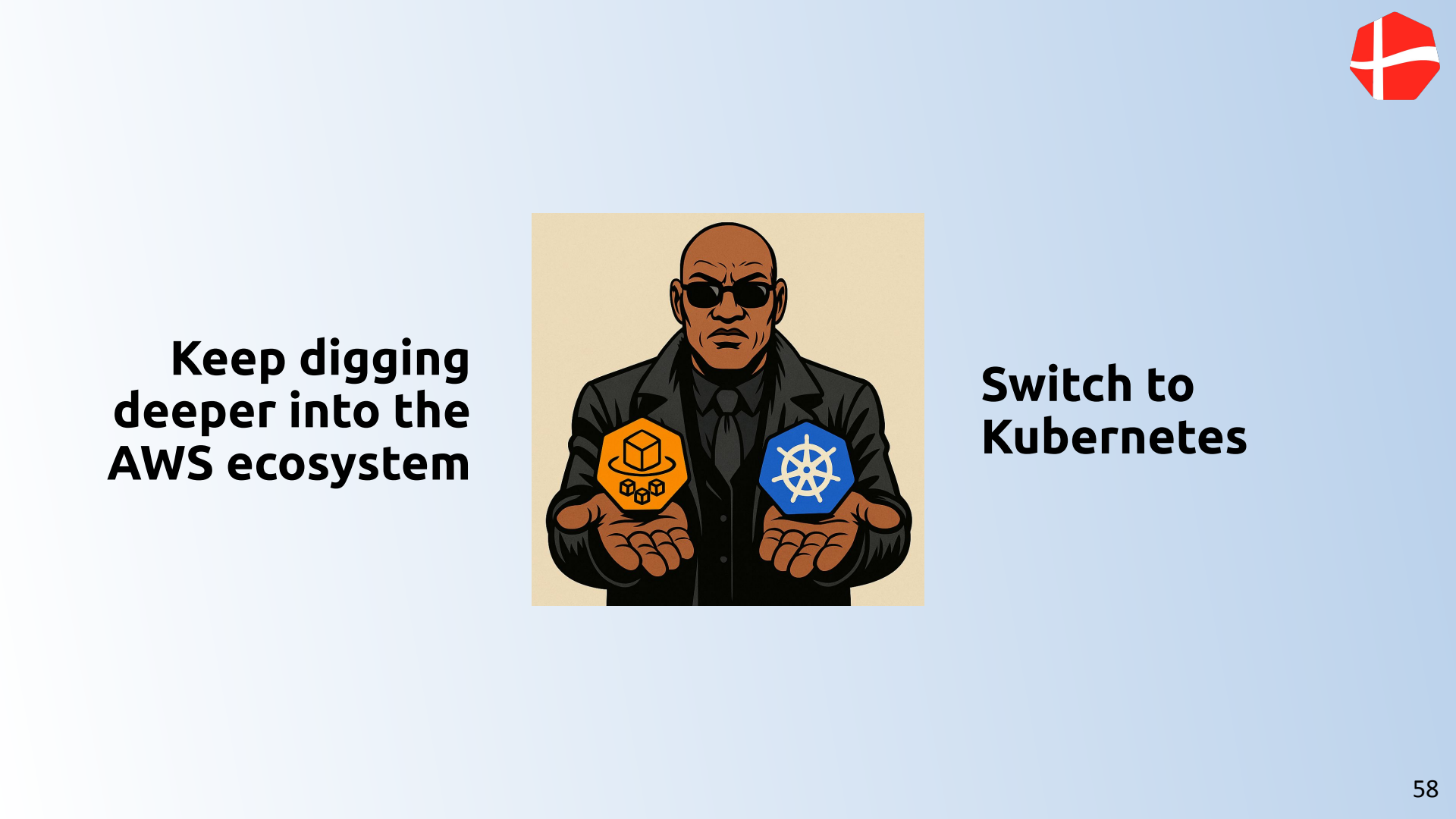
… and this is when it hits you:

# You need a platform

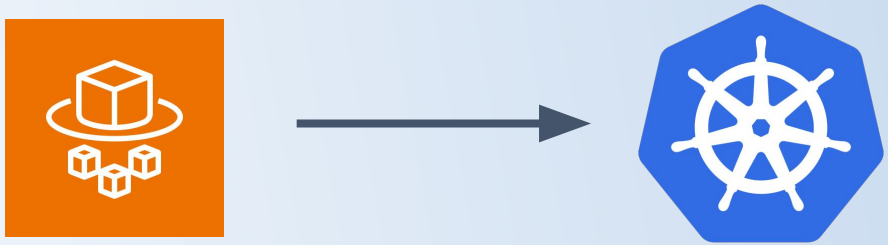

SageMaker
Lambda
EventBridge
Lambda
S3
SQS
SNS
ElastiCache
S3
CloudWatch
Lambda
Fargate
SecretManager


How would you build something complex?
- Tightly coupled to the cloud provider
- A highly customized solution you can’t reuse anywhere else
mbda
S3
Lambda
Fargate
SecretManager


Lambda
S3


Keep digging deeper into the AWS ecosystem
Switch to Kubernetes

# Migrating from AWS Fargate to Kubernetes




# Green.ai

Setting the stage …

Green.ai: Energy Trading & Energy Asset Management

- Algorithmic energy trading
- Balance Energy Assets for Clients

Green.ai

Setting the stage …

Green.ai: Energy Trading & Energy Asset Management

- Algorithmic energy trading
- Balance Energy Assets for Clients

Energy (Trading) Industry:

- lots of external systems
  - That are old/weird/legacy
  - That constantly change
- Almost exclusively machine-to-machine systems
  - Very little room for error (usually none).


Green.ai

Setting the stage …

Green.ai: Energy Trading & Energy Asset Management

- Algorithmic energy trading
- Balance Energy Assets for Clients

Energy (Trading) Industry:

- lots of external systems
  - That are old/weird/legacy
  - That constantly change
- Almost exclusively machine-to-machine systems
  - Very little room for error (usually none).

Start-up:

- We need to adapt and iterate quickly.
- Small team - limited resources.


Green.ai

I will tell you about how we:

Build *the right platform* with K8s


Use K8s to create meaningful abstractions

(Images Generated by ChatGPT)

**Before we build the right platform … how did things look when I arrived at Green.ai ?**

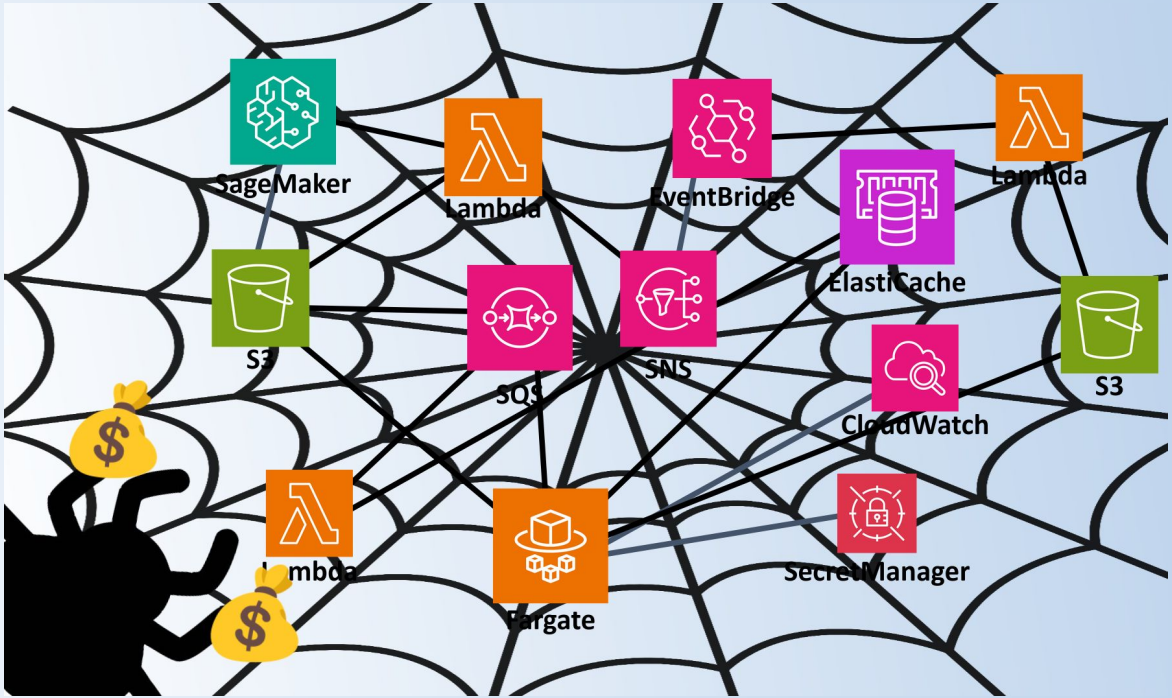
**I joined as employee number 7, they had already been working for ~1.5 years …**

**Before we build the right platform … how did things look when I arrived at Green.ai ?**

**I joined as employee number 7, they had already been working for ~1.5 years …**

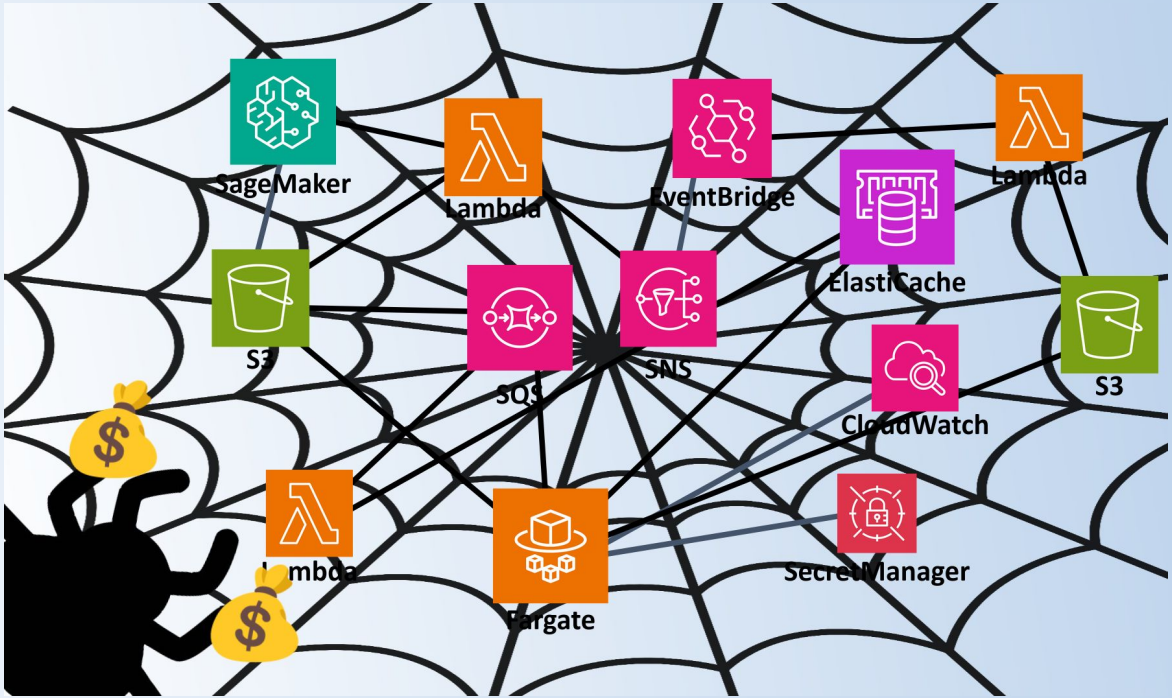
**… and we already had ~~legacy~~ trouble!**

# Another classic start-up journey - to the cloud



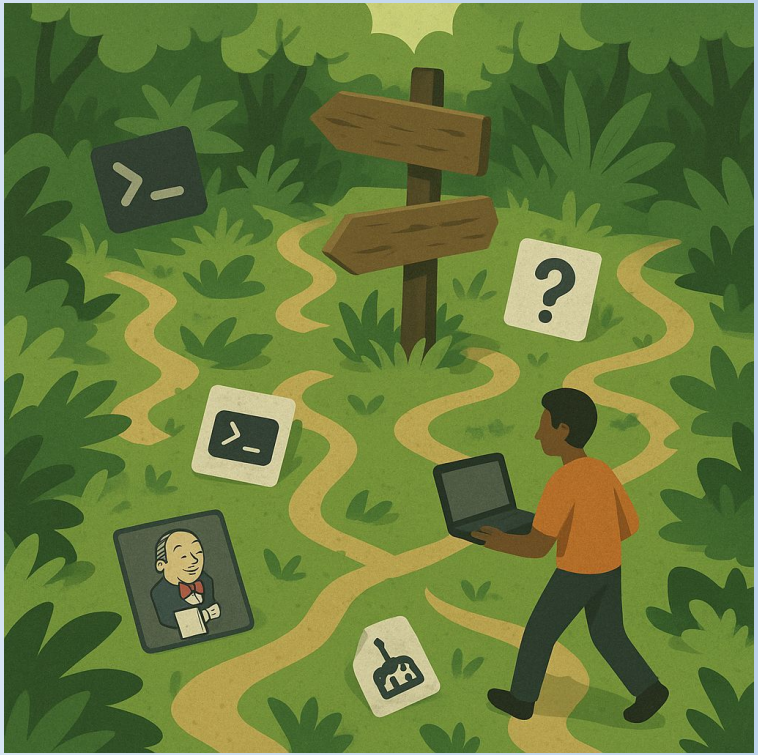
# Another classic start-up journey - to the cloud

… and they were already building a platform ?!



**If you don’t think you are building a platform - you are!**

**What you build and how you do it becomes a platform -**



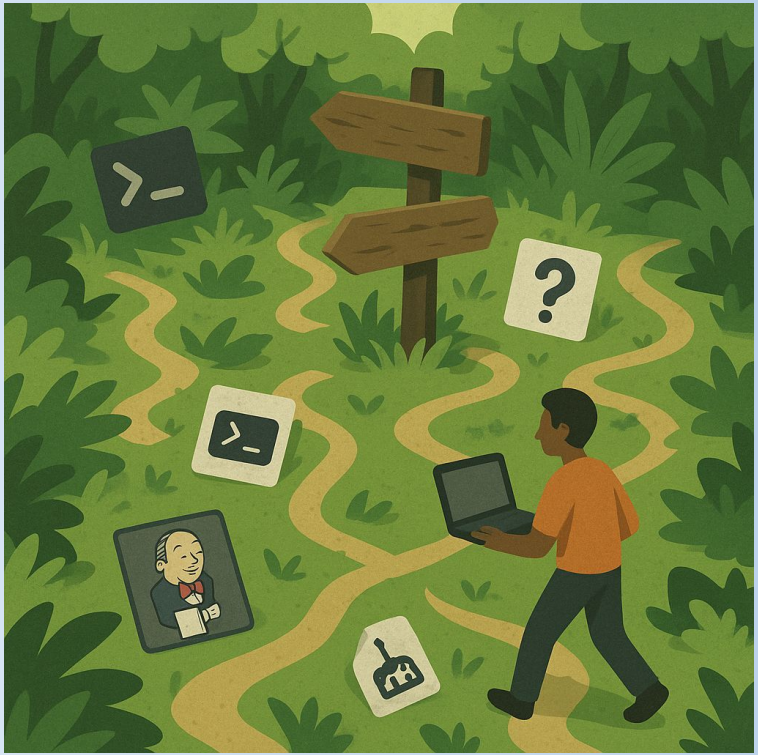
Generated by ChatGPT

**If you don’t think you are building a platform - you are!**

**What you build and how you do it becomes a platform -**

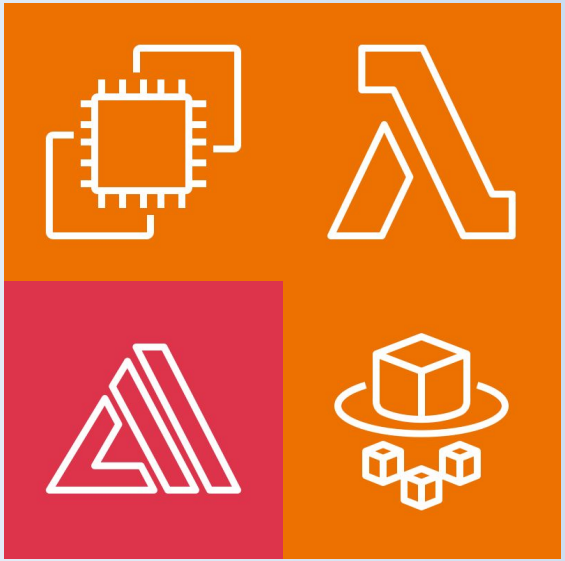
- ***But not in an intentional or structured way***

**I like to refer to this as an *“implicit”* or *“emerging”* platform**



Generated by ChatGPT

# Our Emerging Platform



EC2 (Linux VM)

Lambda (FaaS)

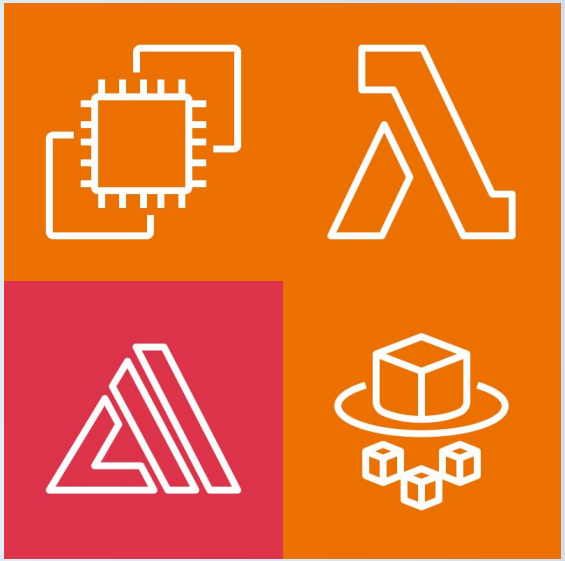
Amplify (serverless frontend)

ECS (CaaS)
(+ Fargate ??)

# 4 Different ways

Manually configure machine, clone git repo and pray

(still don’t know how this works?!)



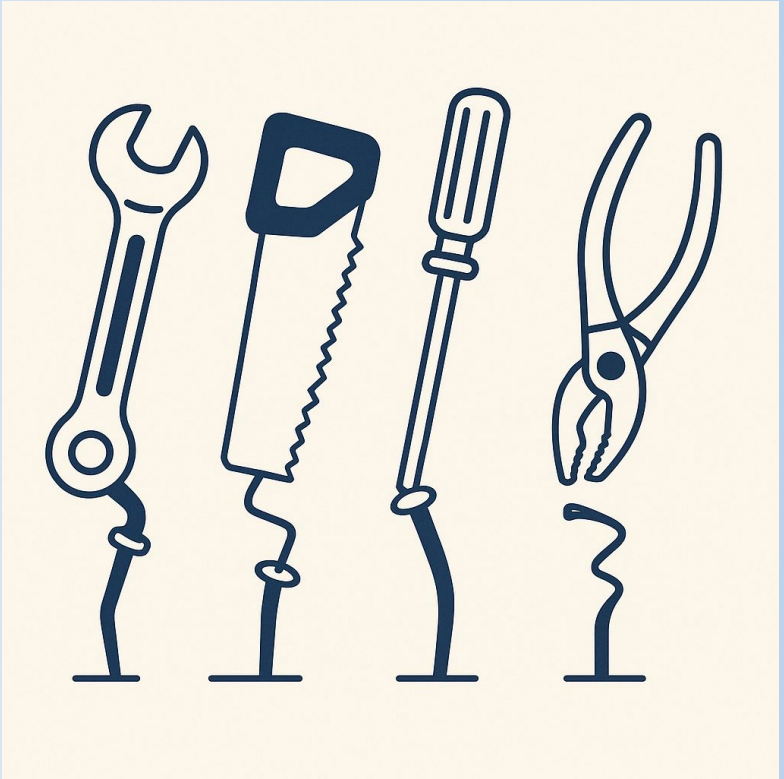
Zappa deploy

Docker + ECR
+ AWS CLI

**Lack of a “perfect solution” lead us to 4 different solutions.**

**This was a pain to work with:**

- **4 different ways of doing things**
- **No structured approach to infrastructure and operations!**
- **Click-ops**



Generated by ChatGPT

# In other words lot’s of complexity!


https://grugbrain.dev

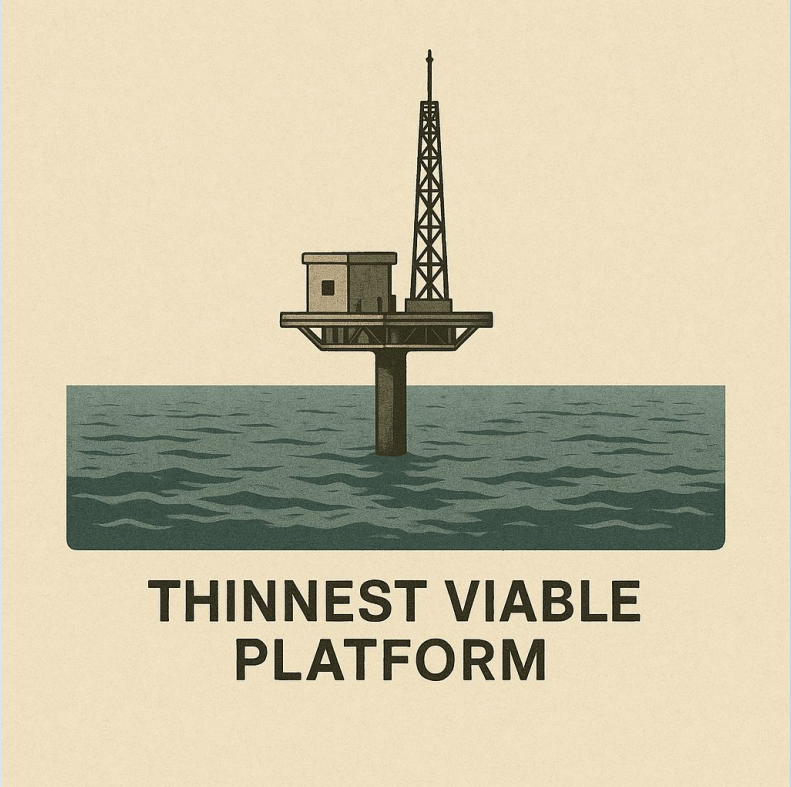
**We decide it’s time for a structured approach to delivering and operating our software.**

… and this is when it hits you:

# You need a platform

# How do you build a platform?

Thinnest Viable Platform (TVP) (Team Topologies)
- *What is the smallest platform we can build?*



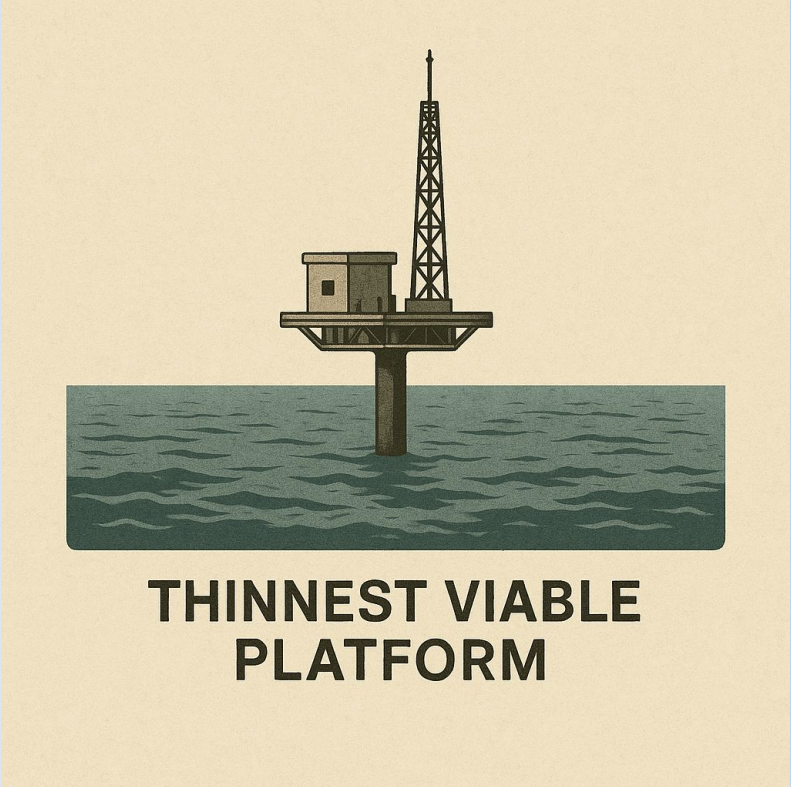

Generated by ChatGPT

# How do you build a platform?

Thinnest Viable Platform (TVP) (Team Topologies)
- *What is the smallest platform we can build?*

We asked these two questions:

- What do we need right now?
- What can we live with doing manually for now and automate later?




Generated by ChatGPT

# Requirements for the TVP:

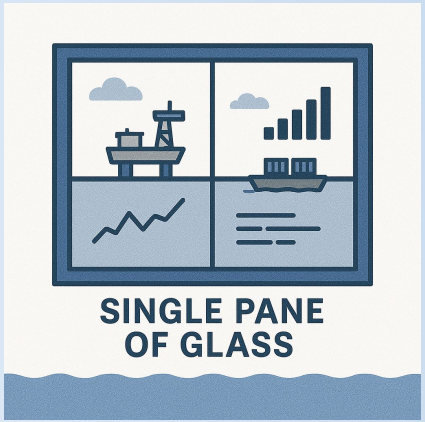

Images Generated by ChatGPT

# Requirements for the TVP:



Images Generated by ChatGPT

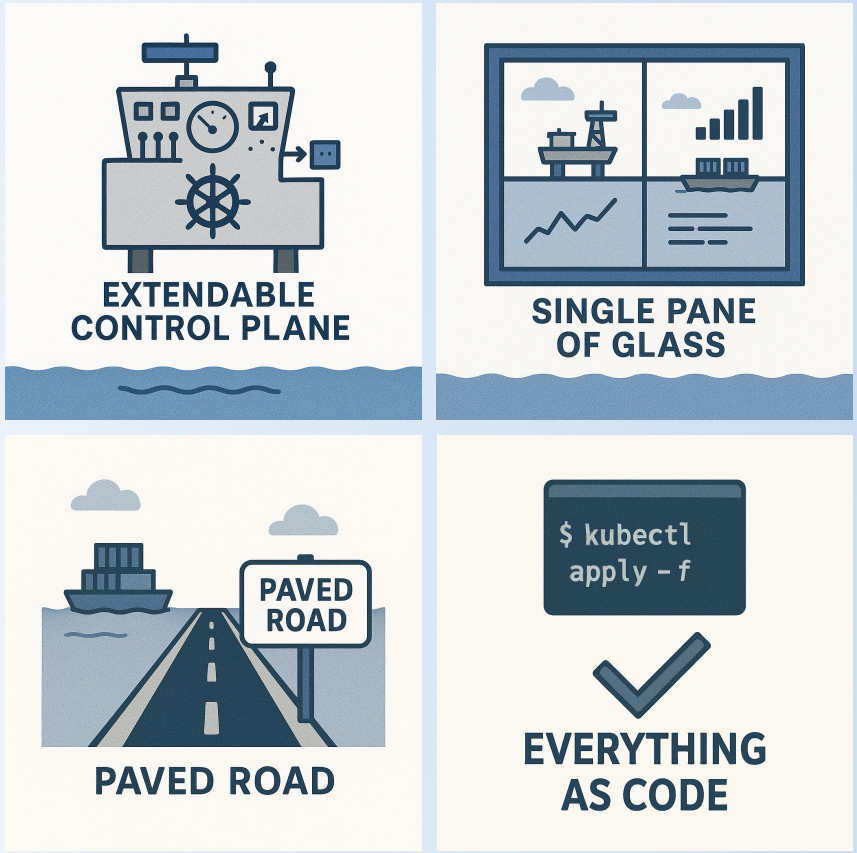
# Requirements for the TVP:



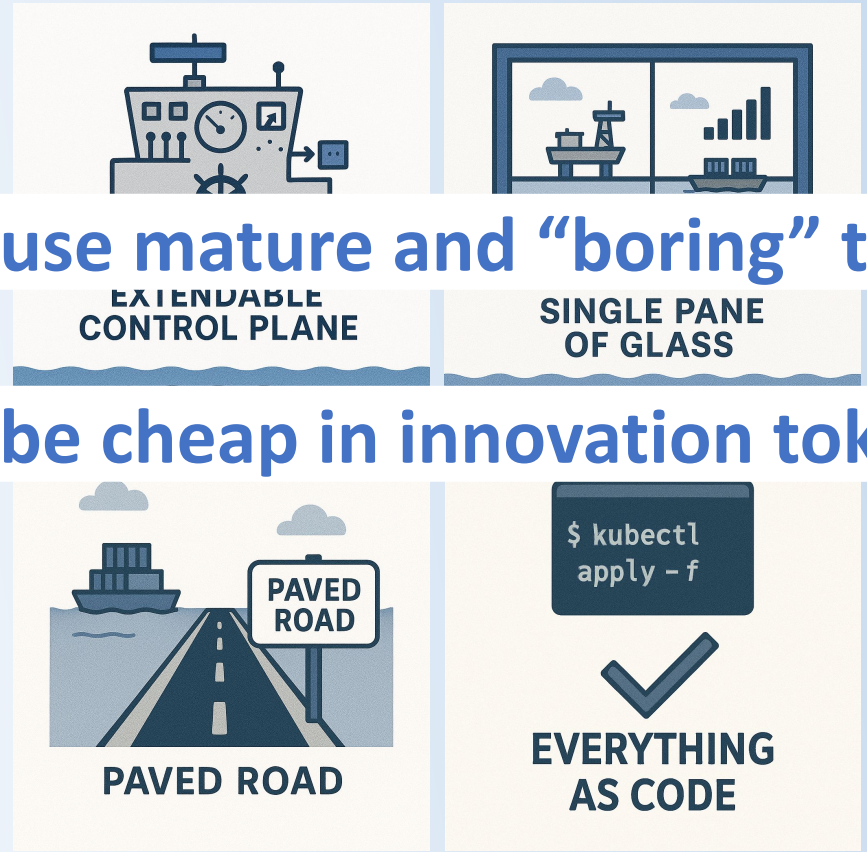
Images Generated by ChatGPT

# Requirements for the TVP:




Images Generated by ChatGPT

# Requirements for the TVP:




+ Must use mature and “boring” technology

+ Must be cheap in innovation tokens !!

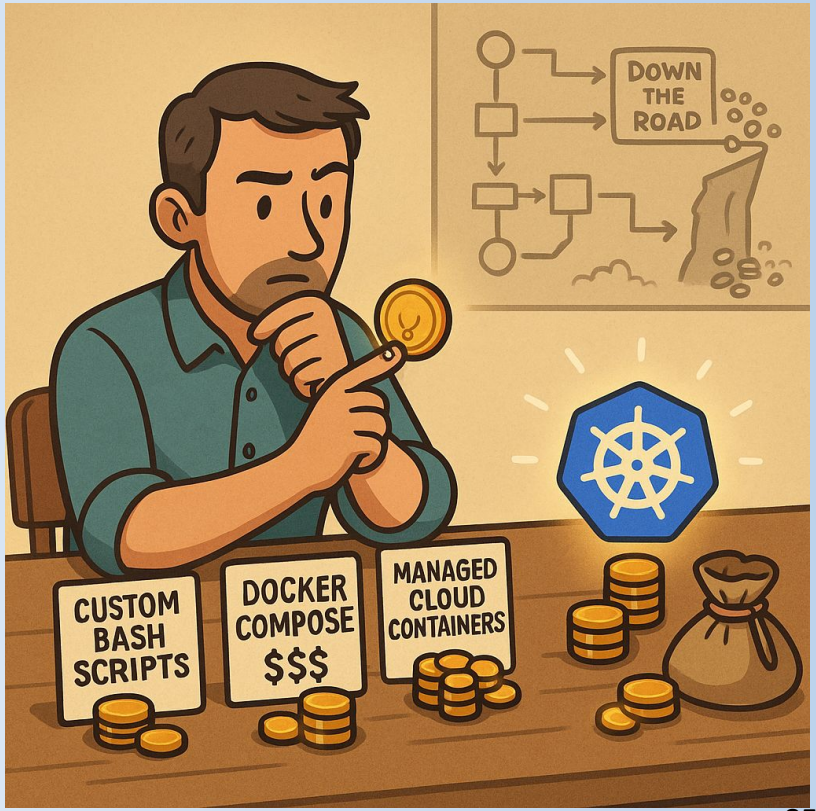
Images Generated by ChatGPT

The only choice:

kubernetes

# How to ~~spend your innovation tokens~~ actually build a platform?

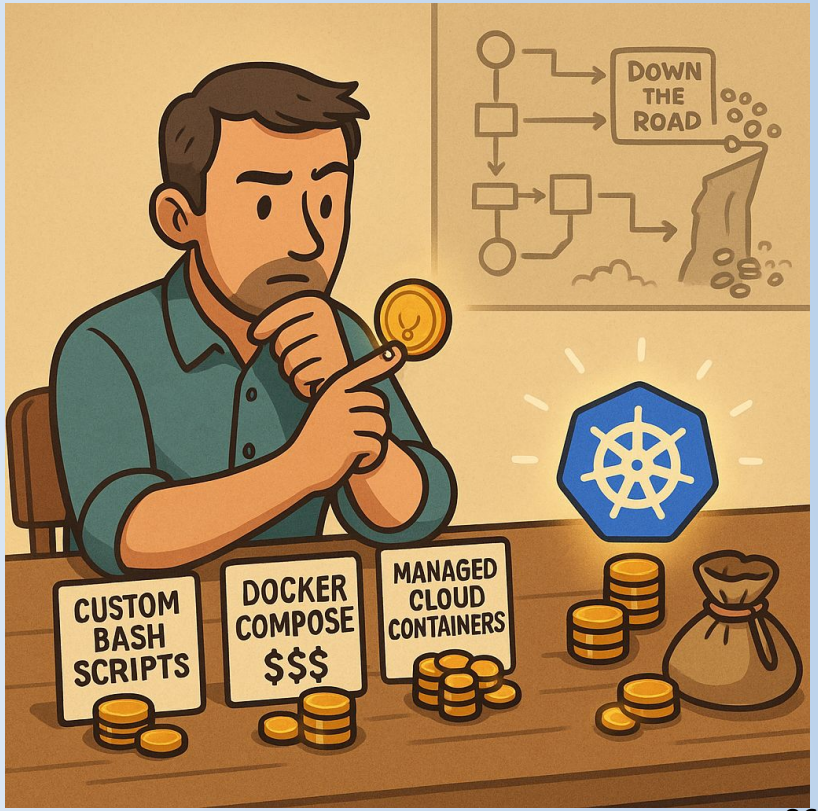
- EKS (Managed K8s)




Generated by ChatGPT

How to ~~spend your innovation tokens~~ actually build a platform?

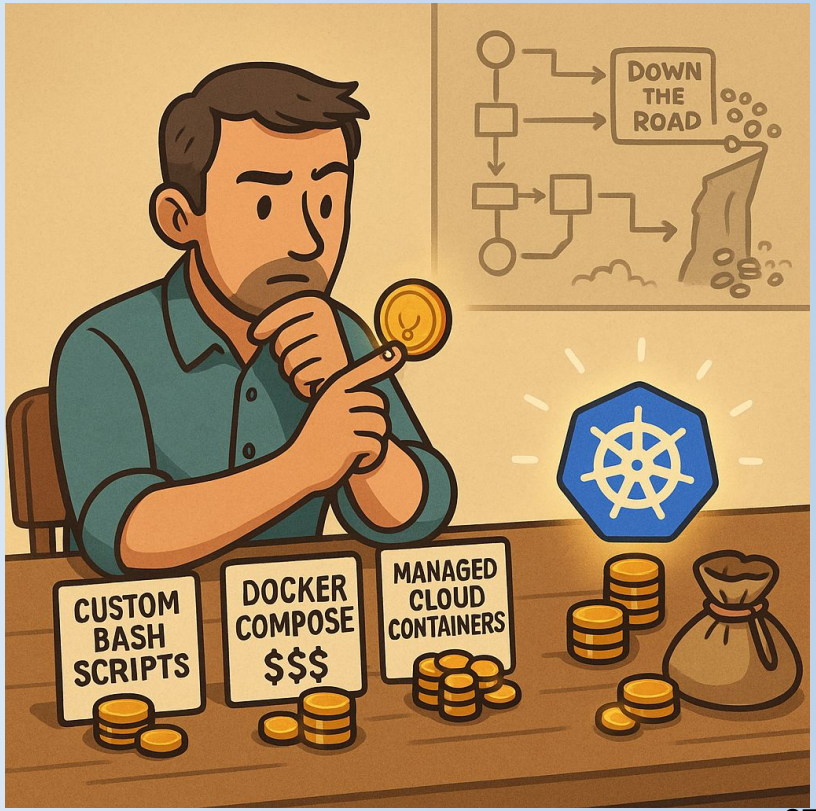
- EKS (Managed K8s)
- Run as many things in K8s as possible




Generated by ChatGPT

How to ~~spend your innovation tokens~~ actually build a platform?

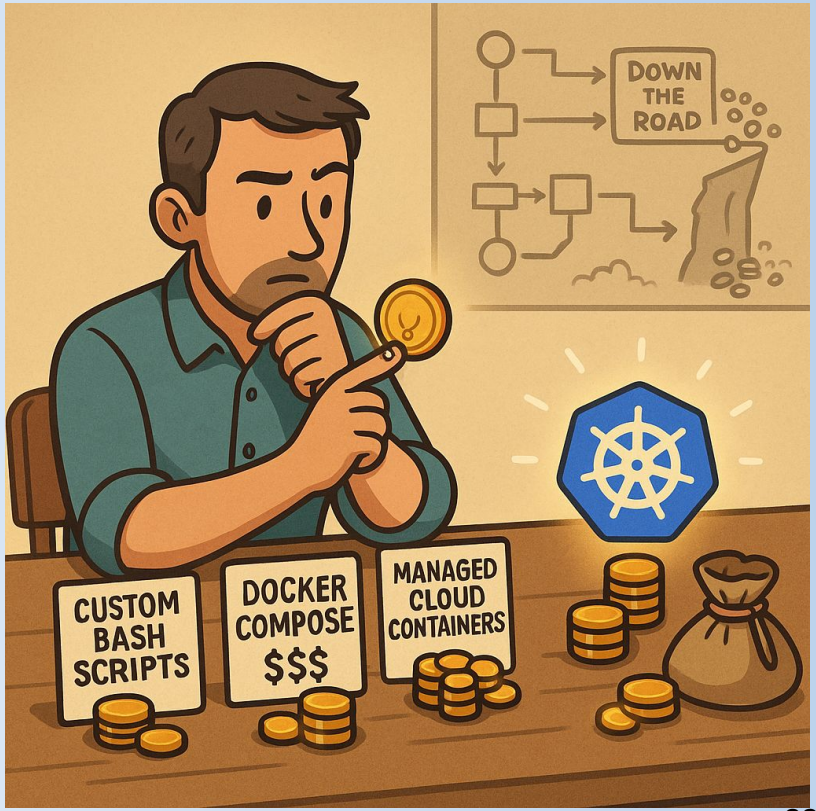
- EKS (Managed K8s)
- Run as many things in K8s as possible
- Let the Controlplane automate things → Controllers!



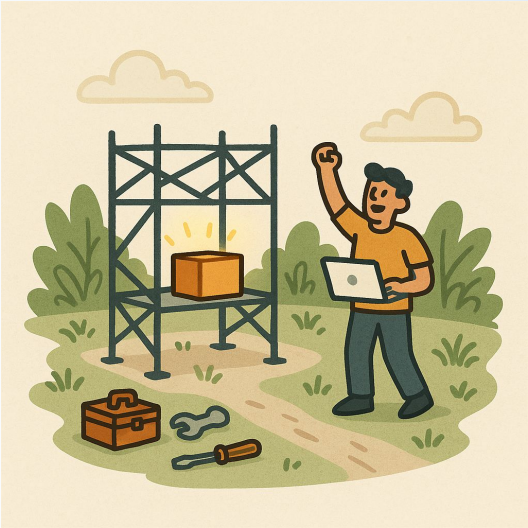

Generated by ChatGPT

# How to ~~spend your innovation tokens~~ actually build a platform?

- EKS (Managed K8s)
- Run as many things in K8s as possible
- Let the Controlplane automate things → Controllers!
- Not everything can run in K8s → use boring tech like OpenTofu/Terraform for the rest




Generated by ChatGPT



First service in production after 4 weeks

Images Generated by ChatGPT

First service in production after 4 weeks


Small/limited initial scope

Images Generated by ChatGPT

First service in production after 4 weeks


Small/limited initial scope


1 way of doing things

Images Generated by ChatGPT

# (in cluster)

Controlplane

Autoscaling

EKS - Managed K8s

Static infrastructure

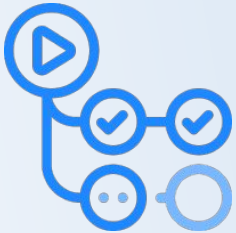
Workflow Orchestration

Controllers:

- Secrets
- DNS
- Ingress
- Storage (CSI)

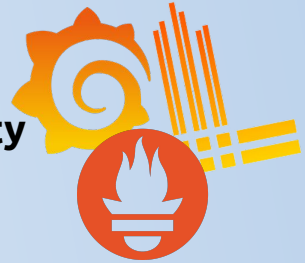

Gitops


Service templating



CI/CD (Github Actions)

Observability



# (in cluster)


Autoscaling

Managed K8s

Static infrastructure

ers:
trets
S
ress
rage (CSI)

Service templating

CI/CD (Github Actions)

bility

… and this is when it hits you:

# We done. We have a platform.

# We have a platform …


https://media1.giphy.com/media/v1.Y2lkPTc5MGl3NjExZjZ5ZG1
0M24wZ3B2OGI3d3Nmd3V4ZmRibmw2cGNwZGFkcTZka2…QZlc
D12MV9pbnRlcm5hbF9naWZfYnlfaWQmY3Q9Zw/c5gKu4…4g/giphy.gif

# Create Meaningful Abstractions


Image Generated by ChatGPT

# Okay, so what's a *meaningful* abstraction?


Image Generated by ChatGPT

**Okay, so what's a *meaningful* abstraction?**

**= Make it easier to solve *business problems*,**
**in your *specific context*,**
**by surfacing the *right complexity*.**


Image Generated by ChatGPT

**In other words, be very specific about the activities that provide value for the business and *how to enable them*!**

# Creating Abstractions - in a startup:

- Start as simple as possible
  - *Add features - when you **need** them!*
  - fx: your helm chart should only template the things you change!

# Creating Abstractions - in a startup:

- Start as simple as possible
  - *Add features - when you **need** them!*
  - fx: your helm chart should only template the things you change!

- Beware of trying to be (too) generic (too) early
  - be *very specific* to your needs
  - You will see *patterns* emerge

# Creating Abstractions - in a startup:

- Start as simple as possible
  - *Add features - when you **need** them!*
  - fx: your helm chart should only template the things you change!

- Beware of trying to be (too) generic (too) early
  - be *very specific* to your needs
  - You will see *patterns* emerge

- Don’t be afraid to (initially) do things ugly and manually
  - it’s all about *iterating - but you must iterate*.

Examples of meaningful abstractions for Green.ai, users* can …

- deploy new trading algorithms.
- deploy and maintain data scrapers.
- expose frontends and APIs.

*Users = Developers + Data Scientists

Examples of meaningful abstractions for Green.ai, users* can …

- deploy new trading algorithms.
- deploy and maintain data scrapers.
- expose frontends and APIs.

- answer questions like …
  - is my service deployed and working?
  - what version is running?
  - did my scheduled workflow succeed?
  - what were the log output?

*Users = Developers + Data Scientists

**None of the examples on the previous slide are unique**

**- but there's a lot of details that goes into realizing them in exactly our context.**

**That's what makes it a meaningful abstraction.**

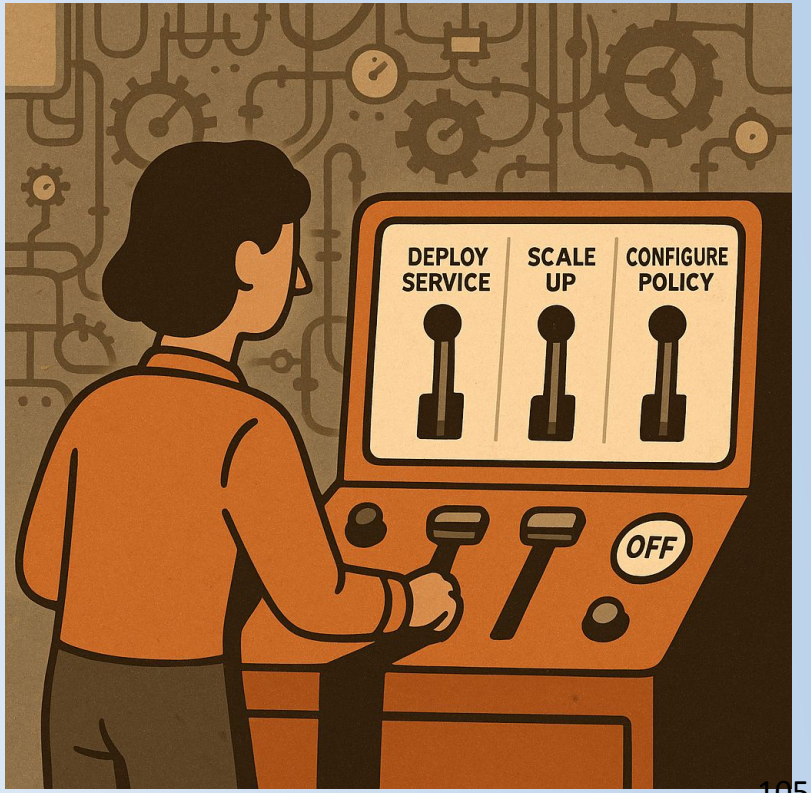



Image Generated by ChatGPT

# Cool, let’s see it in practice.


Image Generated by ChatGPT

```yaml
serviceName: "my-app"
awsRegion: "eu-central-1"
environment: "production"
organization: "green.ai"
domain: "my-domain"
image:
  registry: "123456789.dkr.ecr.eu-central-1.amazonaws.com/my-app"
  tag: "sha-47e0de0"
externalSecrets:
  enabled: true
serviceAccount:
  irsa:
    enabled: true
    iamRoleARN: "arn:aws:iam::123456789:role/irsa-my-app"
deployment:
  enabled: true
  replicas: 2
  resources:
    requests:
      cpu: "0.5"
      memory: 500Mi
ingress:
  enabled: true
  hostName: "my-app.production.green.ai"
  acmCertARN: "arn:aws:acm:eu-central-1:123456789:certificate/ ... "
  oidcAuth:
    enabled: true
```

```
serviceName: "my-app"
awsRegion: "eu-central-1"
environment: "production"
organization: "green.ai"
domain: "my-domain"
image:
  reg                                    /my-app"
  ta
exte
  en
serv
  ir


depl
  en
  re
  re


ingress:
  enabled: true
  hostName: "my-app.production.green.ai"
  acmCertARN: "arn:aws:acm:eu-central-1:123456789:certificate/ ... "
  oidcAuth:
    enabled: true
```

## Abstractions implemented as a Helm Chart

```
serviceName: "my-app"
awsRegion: "eu-central-1"
environment: "production"
organization: "green.ai"
domain: "my-domain"
image:
  reg                                    /my-app"
  ta
exte
  en
serv
  ir


depl
  en
  re
  re


ingress:
  enabled: true
  hostName: "my-app.production.green.ai"
  acmCertARN: "arn:aws:acm:eu-central-1:123456789:certificate/ ... "
  oidcAuth:
    enabled: true
```

**Abstractions implemented as a Helm Chart**

**“Platform interface” - Surfacing the right complexity**

```
serviceName: "my-app"
awsRegion: "eu-central-1"
environment: "production"
organization: "green.ai"
domain: "my-domain"
image:
  reg                                    /my-app"
  ta
exte
  en
serv
  ir


depl
  en
  re
  re


ingress:
  enabled: true
  hostName: "my-app.production.green.ai"
  acmCertARN: "arn:aws:acm:eu-central-1:123456789:certificate/ ... "
  oidcAuth:
    enabled: true
```

**Abstractions implemented as a Helm Chart**

**“Platform interface” - Surfacing the right complexity**

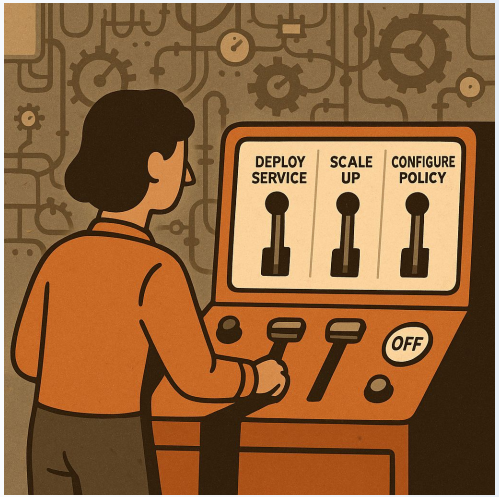
**Helm charts are ugly - and that’s fine!**

**… Helm is very boring technology.**

# Adopting Argo Workflows from a user's perspective:

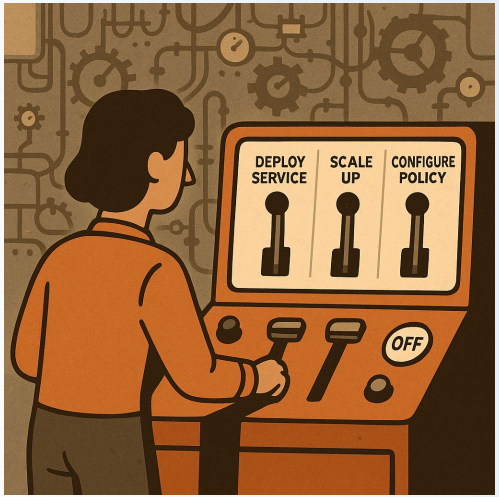
```
62  63          requests:
63  64            cpu: "0.05"
64  65            memory: 110Mi
65      - cronjob:
    66  + cronWorkflows:
66  67      enabled: true
67      -   jobs:
    68  +   cronWorkflows:
68  69        - name:
69  70          schedule: "5 * * * *"
70  71          command:
```

# In summary:




Create meaningful abstractions - focus on the business

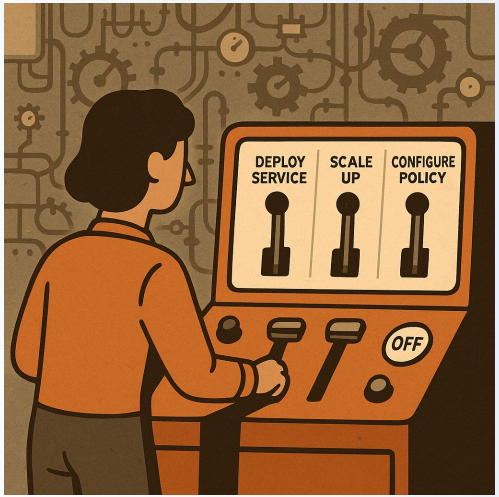
# In summary:




Create meaningful abstractions - focus on the business

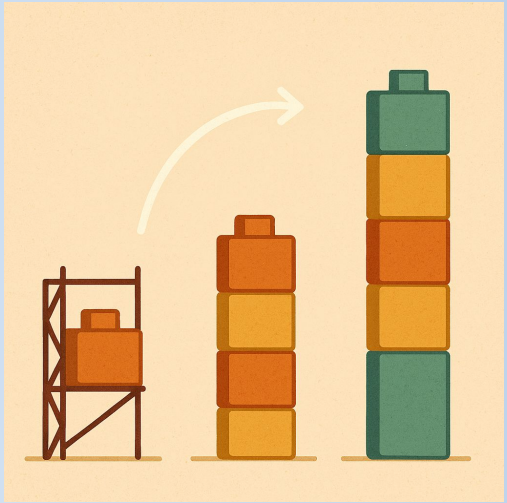

‘platform interfaces’ - standardize and streamline where it matters

# In summary:




Create meaningful abstractions - focus on the business


‘platform interfaces’ - standardize and streamline where it matters



Start small and iterate

# What do we all care about?

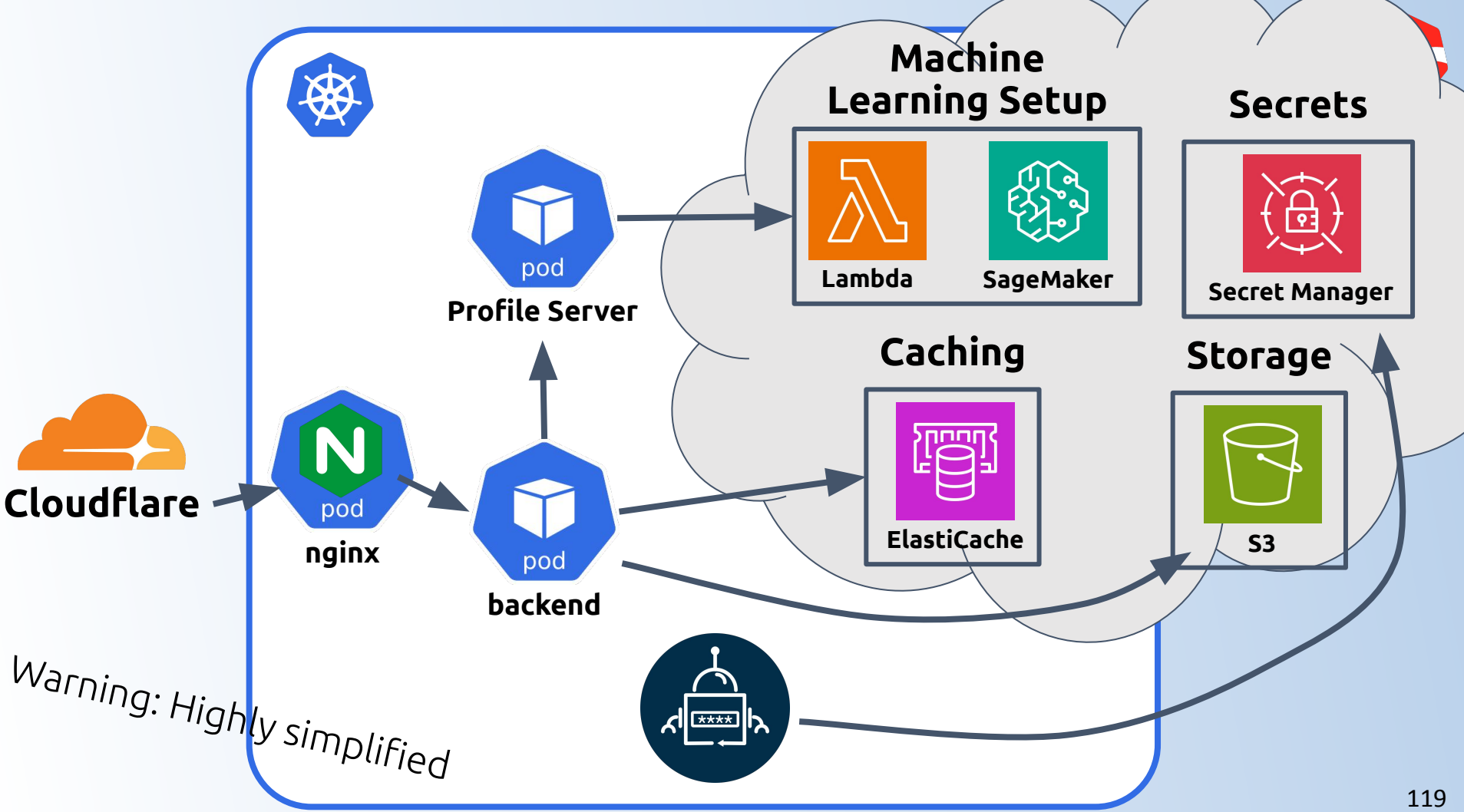
# What do we all care about?

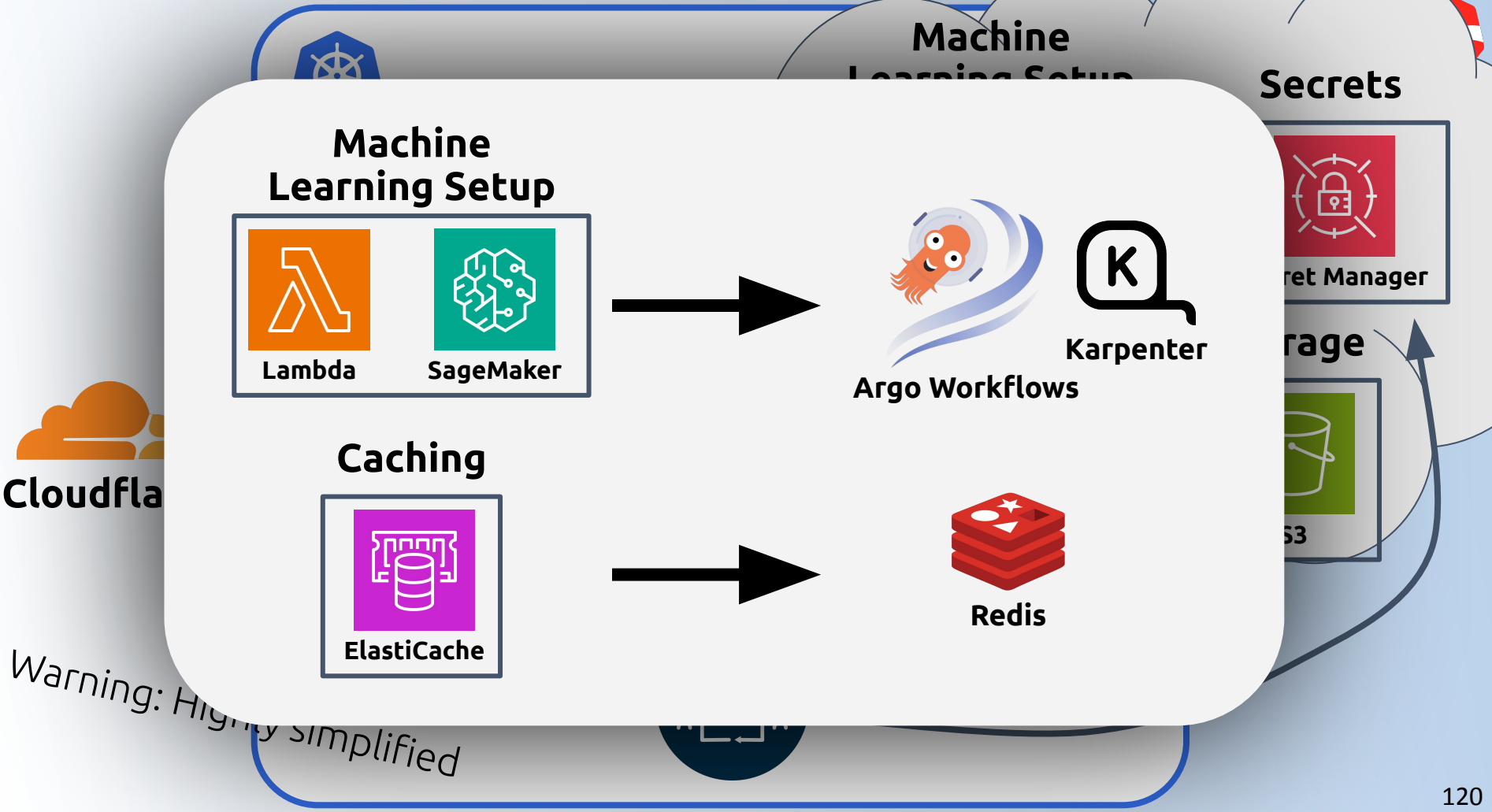
# MONEY

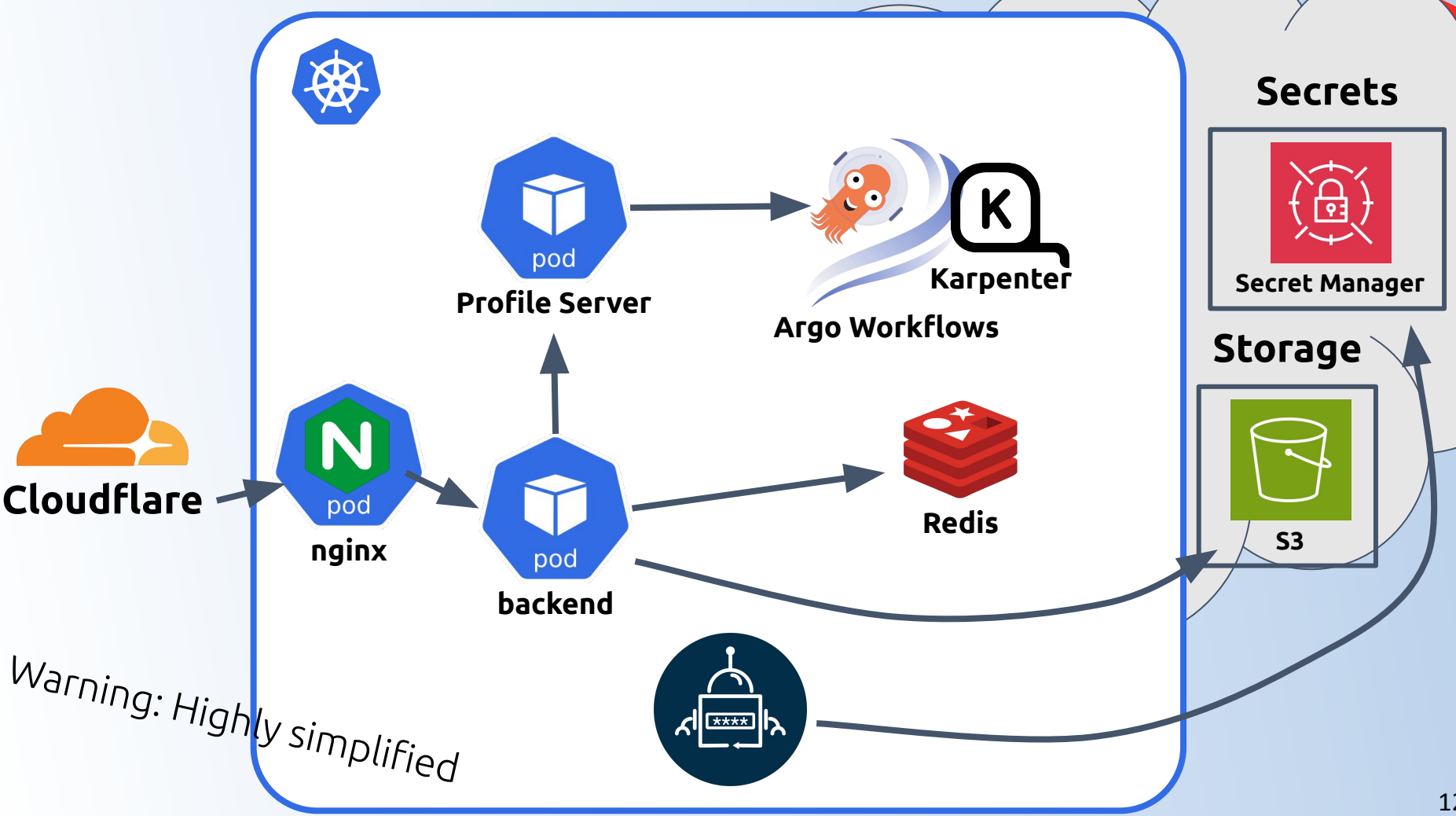
# Migrating from AWS to Azure


# Goal: *Save Money*

**Many companies aim to be “cloud agnostic”, but most of them never make the big switch**


Machine Learning Setup
Secrets
Lambda
SageMaker
Secret Manager
pod
Profile Server
Caching
Storage
Cloudflare
pod
nginx
pod
backend
ElastiCache
S3
****
Warning: Highly simplified

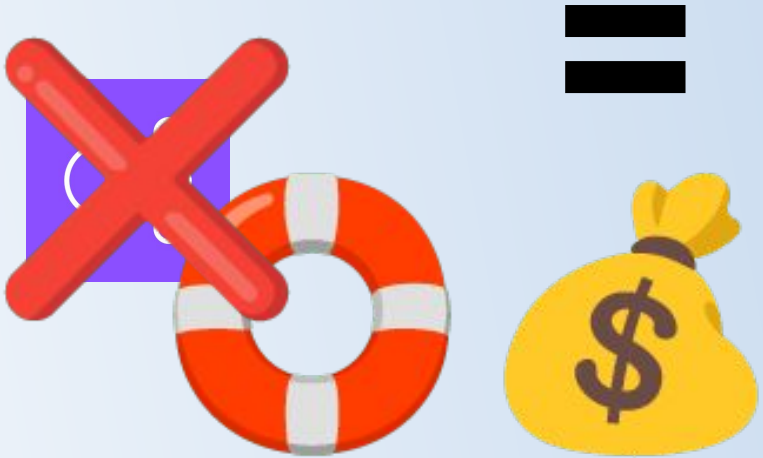

Machine
Learning Setup
Lambda
SageMaker
Argo Workflows
Karpenter
Caching
ElastiCache
Redis
Secrets
ret Manager
rage
S3
Cloudfla
Warning: Highly simplified

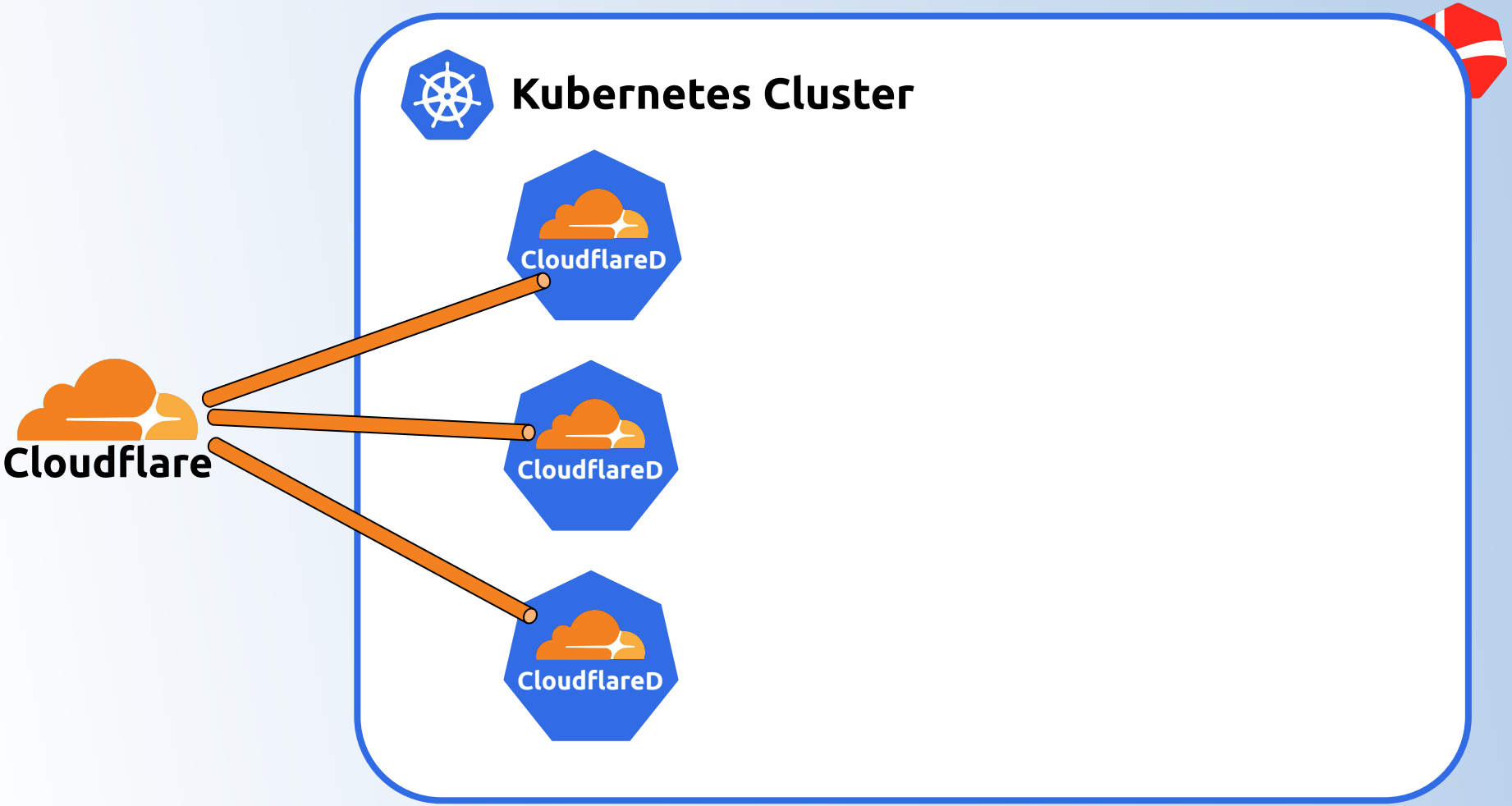

Secrets
Secret Manager
Storage
S3
Cloudflare
pod
nginx
pod
backend
pod
Profile Server
Argo Workflows
Karpenter
Redis
****
Warning: Highly simplified

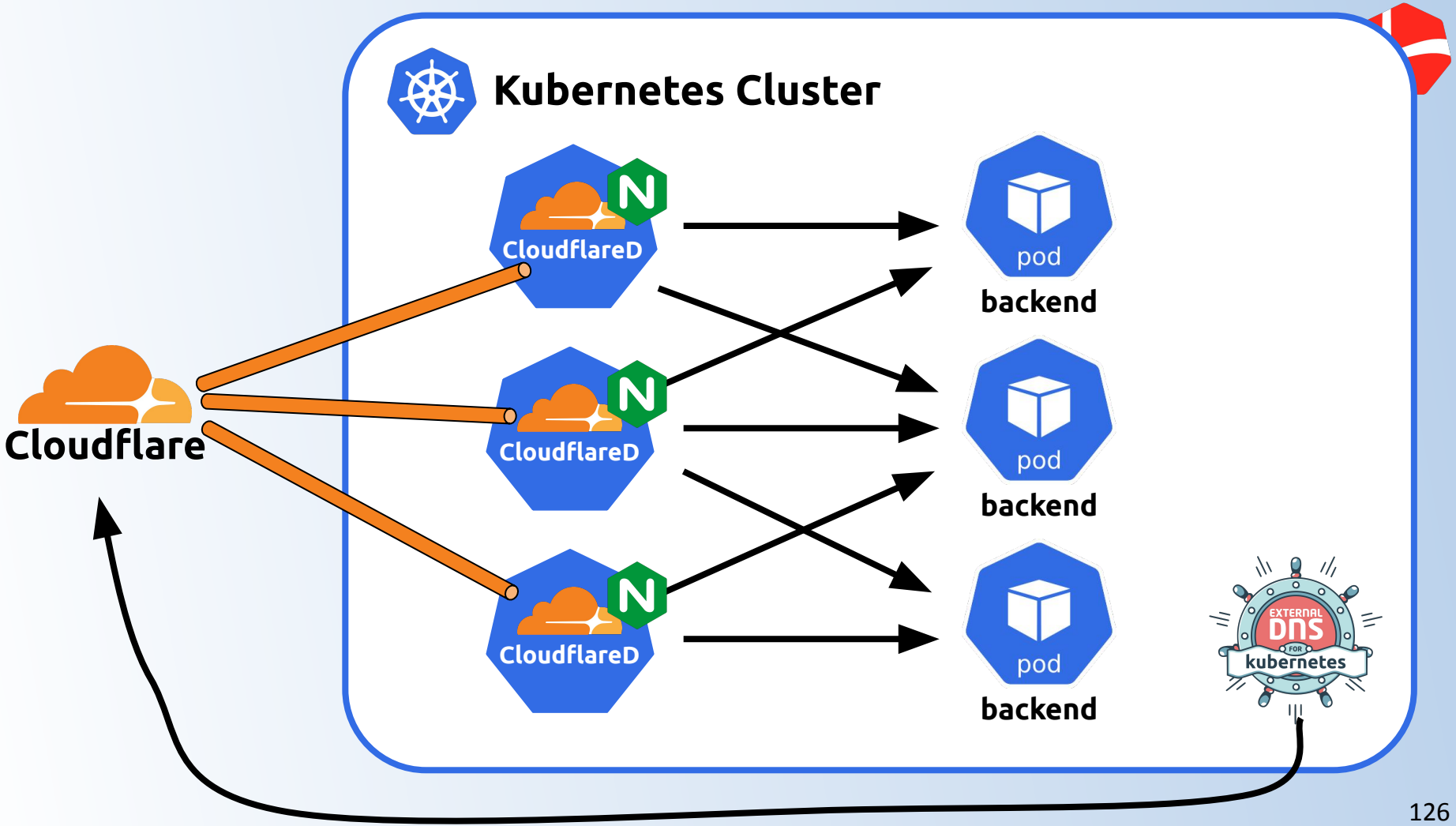
# What about networking and load balancing?

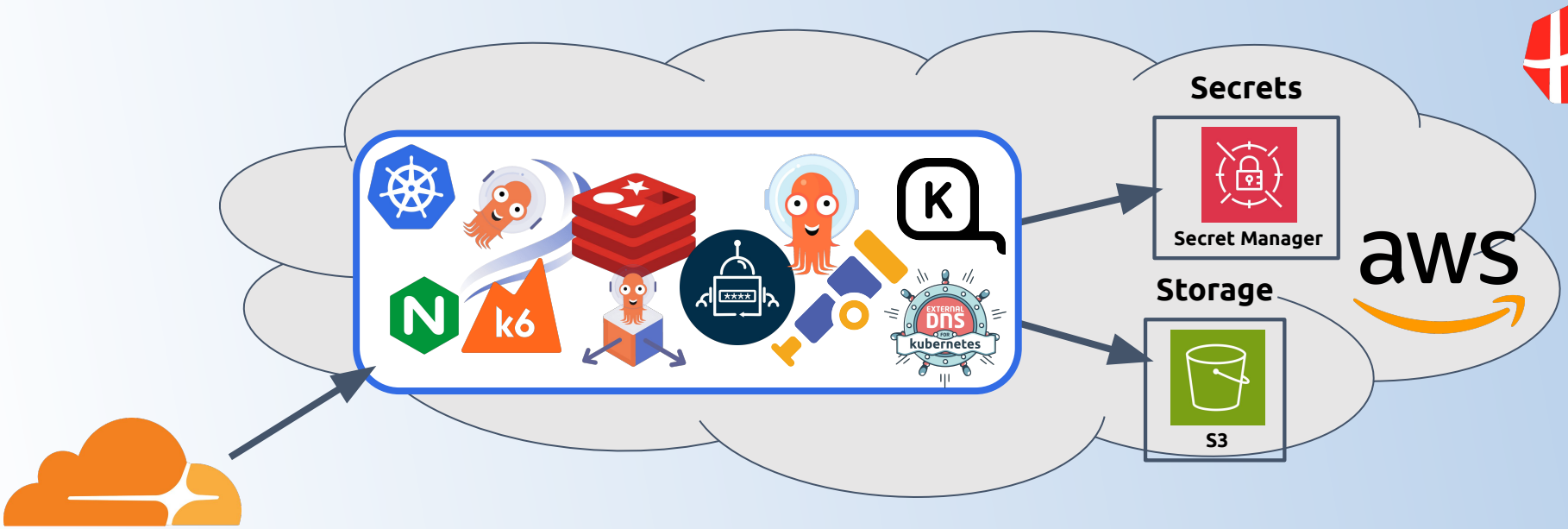
# Load balancer

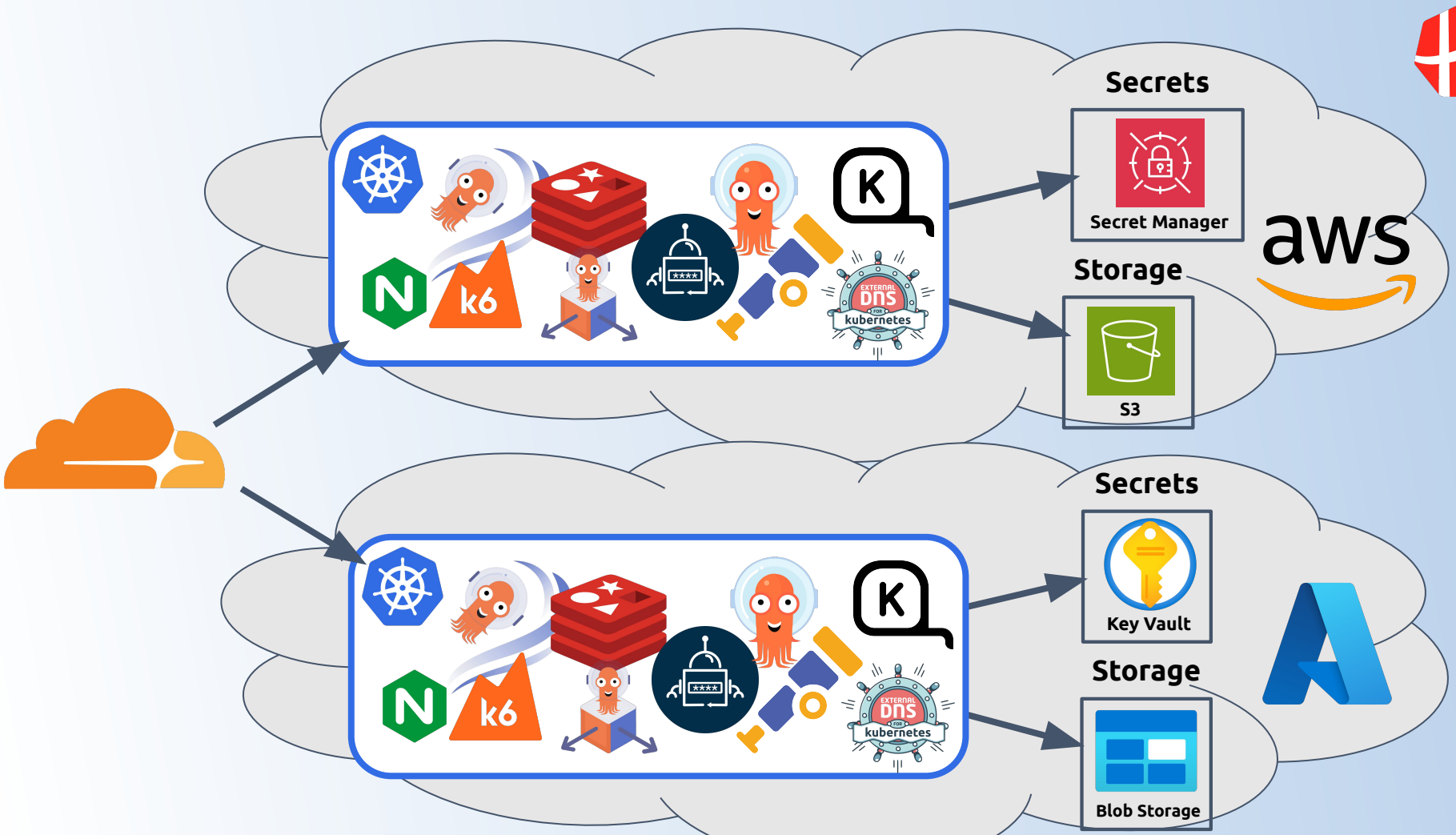
# No load balancer



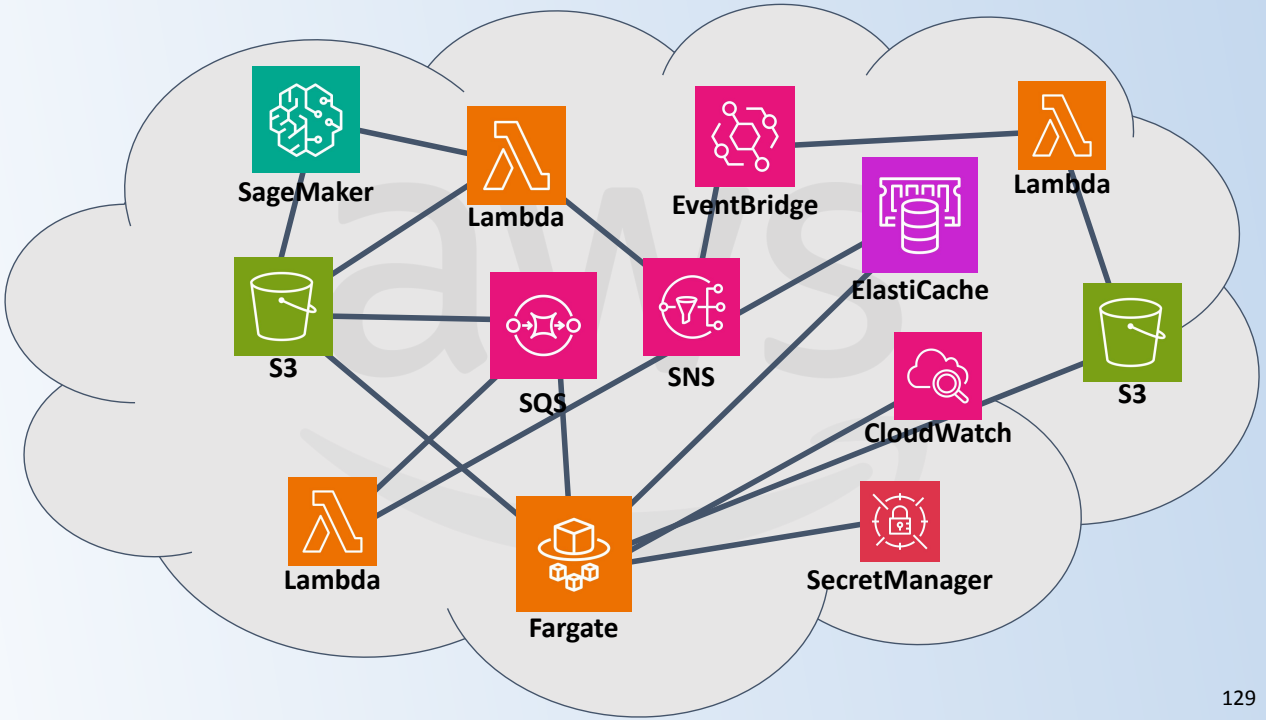

Kubernetes Cluster
CloudflareD
CloudflareD
CloudflareD
Cloudflare


Kubernetes Cluster
Cloudflare
CloudflareD
CloudflareD
CloudflareD
pod
backend
pod
backend
pod
backend
EXTERNAL DNS FOR kubernetes

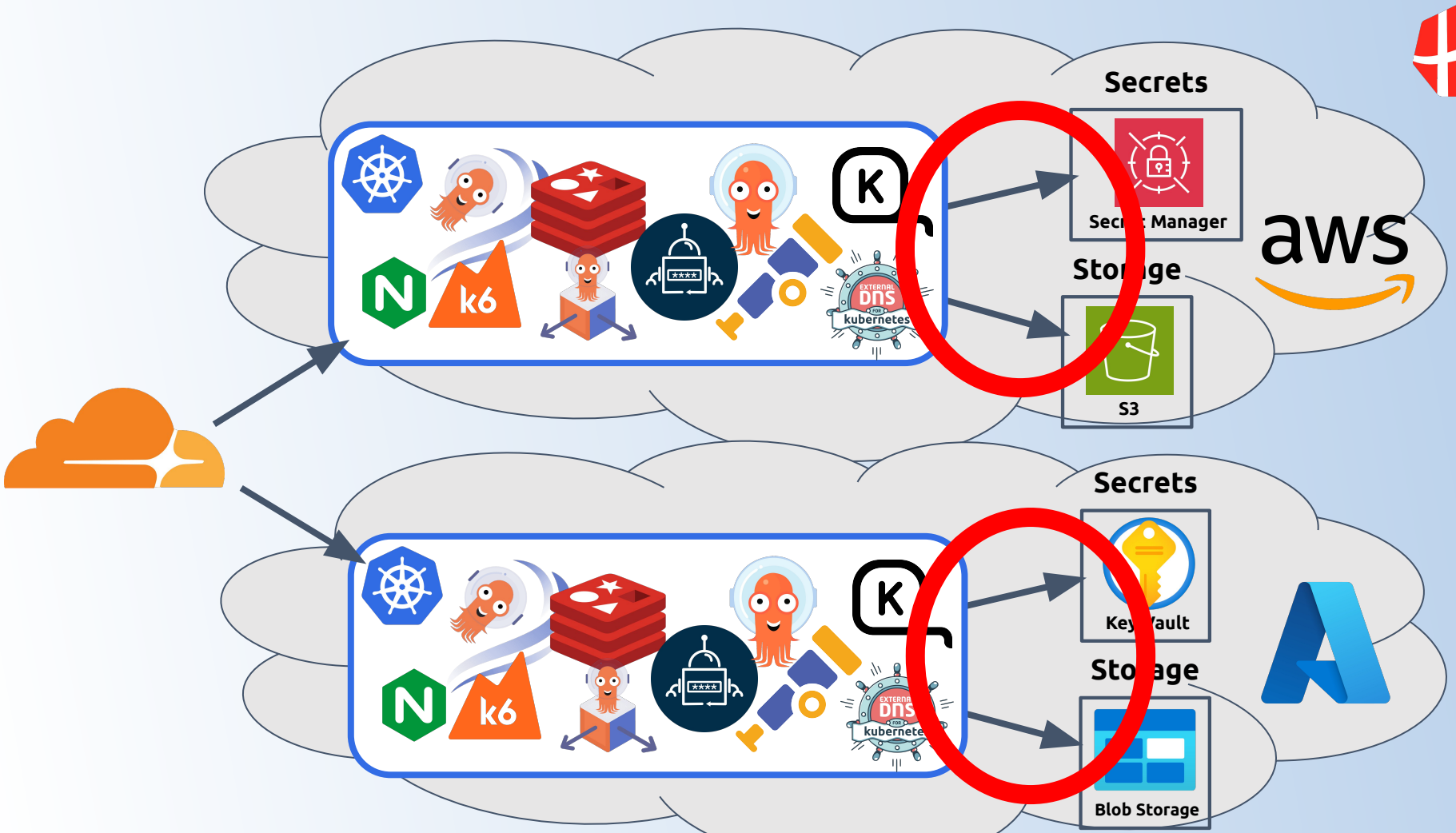

Secrets
Secret Manager
Storage
S3
aws
k6
EXTERNAL DNS
FOR
kubernetes


Secrets
Secret Manager
aws
Storage
S3
k6
kubernetes
Secrets
Key Vault
Storage
Blob Storage

# This would never have worked if we our setup still looked like this



# Was it that easy?


Secrets
Secr Manager
aws
Stor age
EXTERNAL DNS FOR kubernetes
S3
Secrets
Key ault
Sto age
Blob Storage

# Wrap up

# What is the main takeaway from this talk?

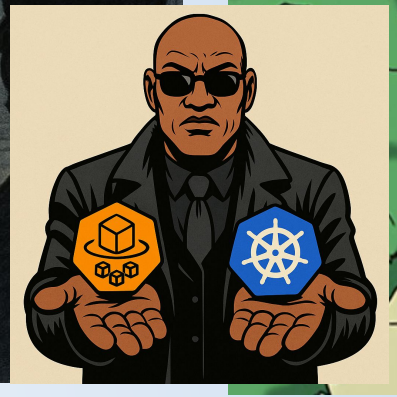
# Your business *is* complex!

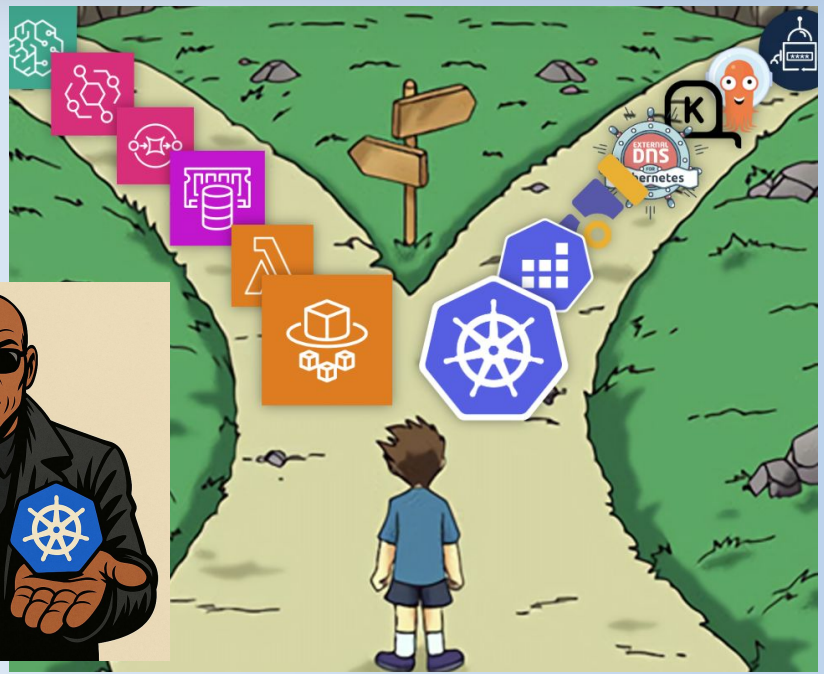
# Developing Software *is* complex!

**Kubernetes is not complex because it *wants* to be,**

**but because it *needs* to be.**

**Serverless is a gateway drug, that is going to lead you down a path you don’t want to take.**





kubernetes

# Kubernetes is a “***boring technology***”

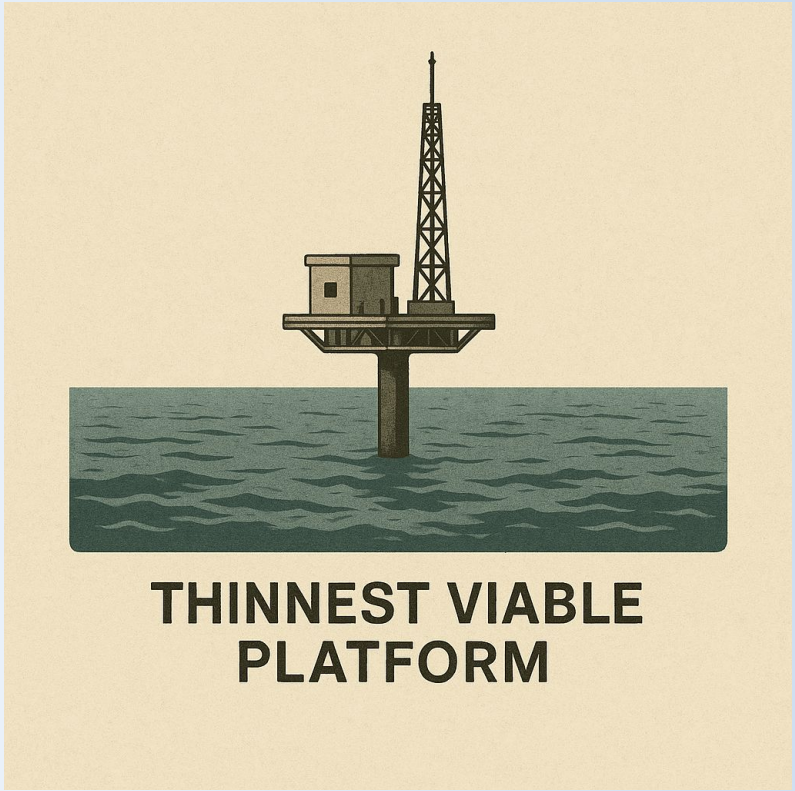
It’s ***well-known***, ***mature***, and has broad ***community and industry support***.

Easier to find documentation.
Easier to debug.
Easier to hire for.
Easier to …

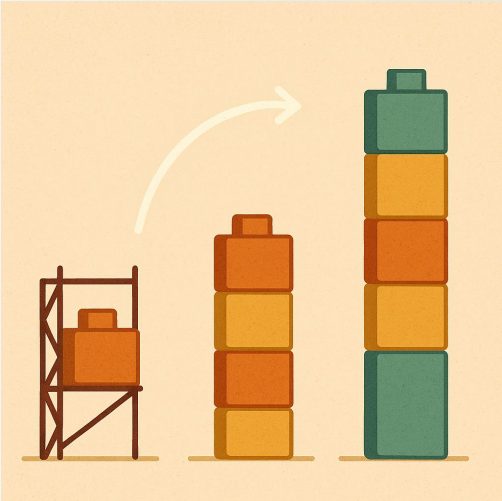

INNOVATION TOKENS

# Picking Kubernetes allows you to switch cloud providers

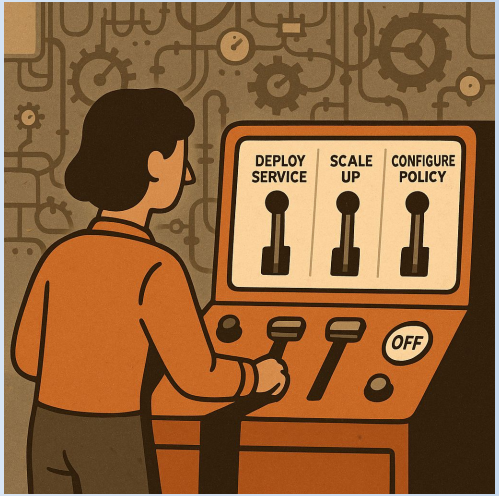

**Goal: *avoid vendor lock-in***


THINNEST VIABLE
PLATFORM

# Focus on building the foundation for future growth



Start small and iterate




Create meaningful abstractions - focus on the business

# This is hopefully also relevant for you (not in a startup??)

# Thank you!

**Zander Hornung Havgaard**
https://www.linkedin.com/in/zanderhavgaard/

**Dag Bjerre Andersen**
https://www.linkedin.com/in/dagbjerreandersen/